

Universidade Pedagógica de Maputo
Faculdade de Engenharia e Tecnologia
Lic. em Informática Aplicada

**Introdução à Programação Orientada a
Objectos**

Tópicos

Unidade 1: Visão geral de linguagens e paradigmas de programação

Unidade 2: Conceitos Básicos de programação (revisão)

Unidade 3: Classes, objectos e métodos

Unidade 4: Herança

Unidade 5: Tratamento de erros (Excepções)

Unidade 6: Introdução à Programação GUI

O autor

Célio Barbosa Sengo é Doutorado em Ciências da Computação (2023) pela Heidelberg University of Education, Alemanha, Mestre em Tecnologias de Informação (2012) pela Punjab College of Technical Education (PCTE) afiliada a Punjab Technical University (PTU), Índia, e Licenciado em Ensino de Informática (2009) pela Universidade Pedagógica de Maputo.

Introdução geral do módulo

Hoje em dia, existem no mercado uma diversidade de aplicações desenvolvidas usando linguagens orientada a objectos, tais como os sistemas de gestão de conteúdos, sistemas de gestão, fóruns de discussão, sem deixar de lado os dispositivos móveis (smartphones/tablets) que na sua maioria usam softwares desenvolvidos nesta abordagem. As linguagens orientadas a objectos trouxeram uma nova forma de desenvolver programas, dando mais atenção aos dados (em detrimento das funções). Actualmente, os telemóveis da SAMSUNG, HUAWEI, SONY, MOTOROLA, possuem o android como sistema operativo, sendo que o android foi desenvolvido pela linguagem Java que é uma linguagem puramente orientada a objectos. Além do Java, existem outras linguagens como o C++, PHP, C#, RUBY, Python, entre outras.

É nesta premissa que o presente módulo visa a discutir os principais conceitos e técnicas de desenvolvimento de software com base nos princípios da programação orientada a objectos. Este módulo, está dividido em 6 unidades:

Unidade 1: Visão geral de linguagens e paradigmas de programação (Ilustração das categorias das linguagens de programação, breve discussão dos princípios fundamentais das linguagens OOP, importância e exemplos de aplicação das linguagens OOP).

Unidade 2: Conceitos Básicos de programação – revisão (breve discussão sobre as variáveis, tipos de dados. Estruturas de controle, funções, matrizes e manipulação de caracteres).

Unidade 3: Classes, objectos e métodos (Objectos, membros, construtores, visibilidade, membros estáticos, sobrecarga, array de objectos e pacotes)

Unidade 4: Herança (Tipos de herança, sobreposição, classes abstractas e pacotes)

Unidade 5: Tratamento de erros – Excepções (Bloco try e catch, Throw e Throws)

Introdução à *Programação Orientada a Objectos*

Unidade 6: Introdução à Programação GUI (Desenho de formulários, botões, labels, caixas de texto, listas, checkboxes, eventos sobre os objectos)

UNIDADE TEMÁTICA I Visão geral de linguagens e paradigmas de programação

Introdução geral

Esta unidade visa a introduzir os princípios básicos de programação orientada a objecto (OOP), onde serão discutidos os principais paradigmas de programação, os elementos básicos do paradigma OOP, bem como a sua importância no quotidiano. O conceito de objecto representa a base para o presente paradigma pois difere do paradigma estruturado onde o problema é convertido em sub programas com dados próprios. Em OOP o problema é transformado em classes e objectos onde deve-se identificar os dados (características) e os métodos (comportamento). Outros princípios são aplicados em objectos como é o caso do encapsulamento de membros em classes, herança que permite a reusabilidade.

Esta unidade está dividida em 2 aulas, onde na primeira aula serão apresentados de forma resumida os principais paradigmas (categorias) de linguagens de programação, tendo como destaque a comparação entre o paradigma OOP e o Estruturado. Na segunda aula serão apresentados os principais elementos de uma linguagem OOP, onde para cada elemento será apresentado um exemplo concreto. Logo no fim será apresentada a importância das linguagens OOP bem como alguns exemplos de aplicação deste paradigma no mundo real.

Por se tratar de uma unidade teórica, serão usados alguns vídeos digitais e websites (artigos) como recursos adicionais de leitura,

LICÃO 1: CATEGORIAS DAS LINGUAGENS DE PROGRAMAÇÃO

1. Introdução

Um software pode ser programado de diferentes formas e técnicas, dependendo do tipo de linguagem de programação usada. Sendo assim, havendo várias linguagens no mercado, é importante perceber de forma elas diferem de modo a saber enquadrar a programação orientada a objectos nestas categorias.

Objectivo geral da aula/Meta

Distinguir os principais paradigmas das linguagens de programação

Objectivos específicos da aula

No fim desta aula o aluno deve ser capaz de:

- Indentificar os principais paradigmas de programação
- Comparar os paradigmas OOP com o Estruturado



2. Os principais paradigmas

Os paradigmas de programação oferecem ao programador uma visão sistemática de como o programa será organizado e executado, onde cada um possui uma técnica diferente. Podemos assim considerar um paradigma de programação como um grupo de linguagens com uma técnica comum.

Alguns paradigmas de programação estão mais centradas na organização do código, outras estão mais focadas em funções/procedimentos.

A seguir iremos discutir de forma resumida os principais paradigmas de programação.

2.1 Paradigma Imperativo

As linguagens de programação deste paradigma baseam-se em comandos que dizem o quê o computador deve fazer. Estes comandos permitem mudar o estado do programa. Deste modo, o maior foco deste tipo de linguagens está mais focada na forma como o programa será executado.

Como exemplo temos as linguagens: FOLTRAN, ALGOL, COBOL, BASIC, PASCAL, C, ADA.



Visite o site wikipedia sobre programação imperativa pela hiperligação:

https://pt.wikipedia.org/wiki/Programa%C3%A7%C3%A3o_imperativa

2.2 Paradigma declarativo

Este paradigma é sempre comparado ao imperativo, pois em vez de direccionar o foco no “como” o programa será escrito, este centra-se no “o quê” o programa deve fazer, isto é, um conjunto de instruções sobre o que se pretende fazer.

Podemos ter como exemplo deste paradigma o XML, SQL, expressões regulares, entre outros.

2.3 Paradigma estruturado

Este paradigma pressupõe a criação de um programa obedecendo à apenas três estruturas: sequência, decisão e iteração. Neste tipo de paradigma o programador concentra-se em transformar o problema em funções e procedimentos, o que levou ao aparecimento do paradigma orientado a objectos. Contudo, ainda hoje existem programadores usando esta técnica, através de linguagens como o C, PASCAL, COBOL, PHP, PERL, GO.

2.4 Paradigma Orientado a objectos

O paradigma orientado a objecto transforma o problema por resolver em diferentes unidades designadas de objectos que interagem entre elas através da troca de mensagens. Comparativamente à programação estruturada, este paradigma agrega as funções/procedimentos e variáveis em objectos, permitindo controlar o seu acesso exterior. Este paradigma está entre os mais usados na actualidade, tendo ganho forças com linguagens como o Java, C#, C++ que vem dominando o mercado. Mas detalhes serão discutidos na próxima aula (Princípios fundamentais OOP).

2.5 Paradigma funcional

Este paradigma centra-se em olhar o programa de computador como uma lista de funções matemáticas, onde os parâmetros são os valores de entrada/leitura.

Este tipo de paradigma é geralmente usado várias áreas como a estatística e análise financeira. Como exemplo de linguagens funcionais temos o LISP, Haskell, OCaml, F#, LOGO, entre outras.

2.6 Outros paradigmas

Existem ainda outros que diferem dos que foram discutidos aqui, como é o caso do paradigma lógico, o paradigma multi-paradigma, entre outros.

3. Comparação: Paradigma Estruturado vs Orientado a Objectos

A programação orientada a objectos trouxe uma nova dinâmica no processo de desenho de softwares em detrimento da estruturada. As linguagens estruturadas centram-se na divisão do problema em partes com funcionalidades específicas, usam a abordagem top-down, focam-se em produzir instruções para resolver o problema e não permitem esconder dados. Por outro lado nas linguagens orientadas a objectos o problema é modelado em objectos que possuem dados e métodos que são protegidos. O mais importante é identificar os objectos e as mensagens entre eles. Os programas são mais simples e de fácil perceber pois qualquer coisa que existe no mundo é uma representação de objectos. É desta forma que a presente actividade visa moldar aos estudantes as várias diferenças que existem entre estas duas linguagens...

A tabela abaixo , apresenta de forma resumida as principais diferenças entre as duas abordagens:

Paradigma Orientado a Objectos	Paradigma Estruturado
Usa as seções de um programa para executar determinadas tarefas. Ele divide o programa em objetos que podem ser reutilizados em outros programas . Um programa orientado a objeto é decomposto em uma rede de objetos de colaboração. Para qualquer atividade do programa, um objeto responsável por essa atividade pode interagir com outros objetos, invocando os seus comportamentos, ou "métodos", até que a atividade seja	Assume a abordagem top-bottom. É baseada em torno de estruturas de dados e rotinas. Ele divide as tarefas em formas modulares. Realiza determinadas tarefas para que de uma razão específica. Um programa estruturado é decomposta em uma hierarquia de processos.

concluída.	
------------	--



a) Programação Estruturada vs Programação Orientada a Objectos (Dr K R Bond 2000)

Resumo: Este texto fala sobre o surgimento da programação orientada a objectos e faz um estudo comparativo das duas abordagens olhando para os seguintes aspectos: estrutura top - down, programação modular, resolução de problemas, complexidade das intruções e compilação.

b) Referência completa: The Computer Revolution/Programming/Object Oriented vs. Structured programming

http://en.wikibooks.org/wiki/The_Computer_Revolution/Programming/Object_Oriented_vs._Structured_programming

Resumo: Este website oferece um resumo sobre as principais diferenças entre as linguagens estruturadas e as linguagens orientadas a objectos.

LIÇÃO 2: PRINCÍPIOS FUNDAMENTAIS EM OOP

1. Introdução

Como foi discutido na aula anterior, uma linguagem orientada a objectos as instruções são agregadas em unidades chamadas de objectos. Esta, não constitui a única característica das linguagens OOP, existindo outras, com designação de princípios. Assim, as linguagens OOP possuem os seguintes princípios: Classes, Objectos, Herança, Encapsulamento, Abstracção, Polimorfismo e Mensagem.

Assim, a presente aula visa discutir com exemplos claros os principais princípios/características de uma linguagem OOP, bem como os exemplos da aplicação deste tipo de linguagens.

Objectivo geral da aula/Meta

No fim desta aula o aluno deve ser capaz de:

- Indentificar os principais princípios de uma linguagem OOP
- Relacionar cada um dos princípios à exemplos ligados ao cotidiano
- Compreender a importância e áreas de aplicação das linguagens OOP



2. Princípios fundamentais OOP

Para que uma linguagem seja considerada como “orientada a objectos” deve suportar/obedecer os seguintes princípios:

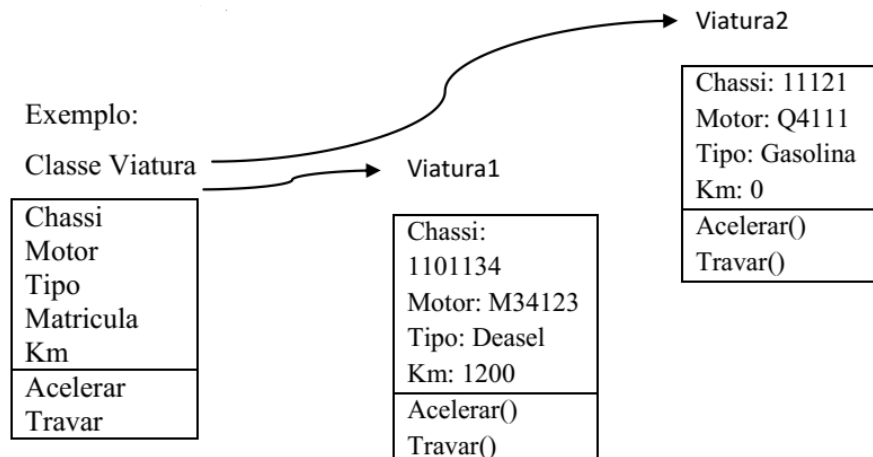
- Classes
- Objectos
- Abstracção e Encapsulamento
- Herança
- Mensagem
- Polimorfismo

A seguir iremos discutir cada um destes elementos, dando ênfase a dois aspectos: a sua característica e exemplos concretos:

2.1 Classes e Objectos

Quando pretendemos construir um carro, o primeiro passo consiste em desenhar o modelo do carro, onde através dele, poderemos construir os restantes. Em

rogramação orientada a objectos iniciamos com o desenho do modelo de algo no qual nos interessa manipular no sistema, a classe. E cada um dos carros que forem produzidos através deste modelo, constitui uma instância do modelo criado. Deste modo, para cada classe é possível criar vários objectos, onde todos objectos têm características comuns, contudo, cada objecto possui suas próprias especificidades. Todas as classes possuem dados e métodos, onde os dados são a descrição da classe (variáveis) e os métodos constituem o comportamento, isto é, funções que manipulam os dados. Se fizermos uma analogia do carro, o acelerar por exemplo implica alguma mudança na quilometragem que vai aumentar.



Podemos notar aqui que sobre a classe viatura foram criados dois objectos Viatura1 e Viatura2. Todas as viaturas (por pertencerem a mesma classe) possuem as mesmas características, contudo, cada viatura possui suas próprias características.

2.2 Abstracção e encapsulamento de dados

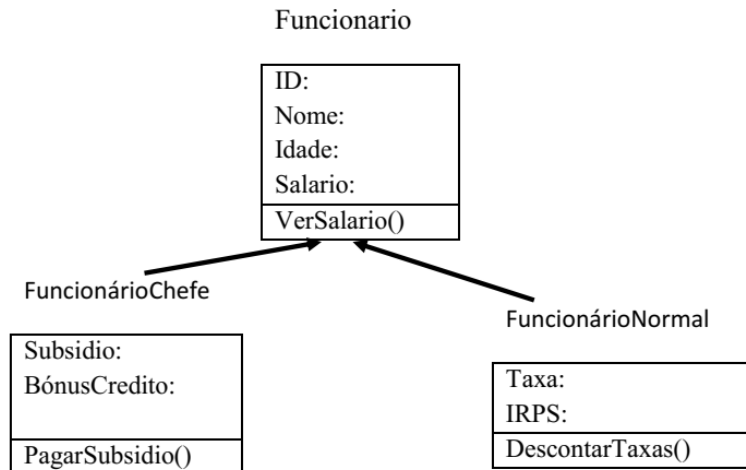
Em OOP todos os membros da classe (dados e métodos) devem estar contidos dentro da classe. Este processo é chamado de encapsulamento, onde dados e métodos são embutidos em classes de modo que estejam protegidos do acesso exterior. O encapsulamento está relacionado à abstracção que consiste no uso dos métodos ou dados duma classe sem se preocupar com os seus detalhes ou características internas. Se fizermos uma analogia com o carro, o condutor prime o acelerador sem precisar saber dos detalhes mecânicos internos por detrás do pedal. Daí nos podemos fazer o uso de métodos duma classe por exemplo sem precisar saber do código interno deste.

2.3 Herança

A herança consiste na habilidade de uma classe herdar as características de uma outra. Este conceito é muito importante, pois permite-nos fazer o uso dos métodos/dados de uma classe, sem necessariamente termos que reprogramar, mas sim, reutilizar o que já existe.

Introdução à Programação Orientada a Objectos

Na herança, uma classe herda a outra sem modificar o seu conteúdo, onde a classe que herda é chamada de sub classe ou classe derivada, e a classe que é herdada é chamada de super classe ou classe derivada.



A classe base funcionário possui duas classes derivadas **FuncionarioChefe** e **Funcionario Normal**. Tanto o normal como o chefe, possui ID, Nome, Idade e Salário, porém, o funcionário Chefe possui também Subsidio, BónusCrédito e PagarSubssidio. Portanto, as classes derivadas herdam os métodos e dados da classe base além de possuírem suas próprias características.

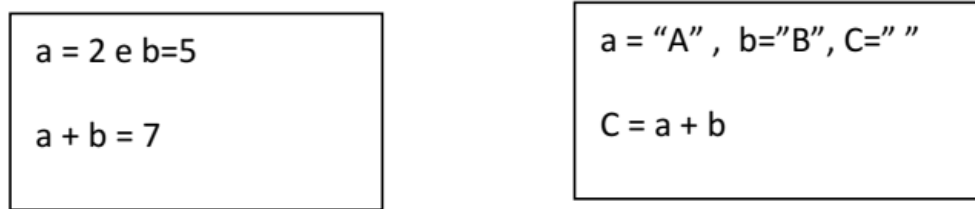
Este conceito será discutido na unidade III.

2.4Polimorfismo

De forma geral, o polimorfismo é muito usado com a finalidade de permitir que alguma coisa qualquer possa assumir mais de uma forma, dependendo da situação concreta. Em OOP, quando duas ou mais subclasses invocam métodos com o mesmo nome mas com comportamentos diferentes, estamos perante o polimorfismo. Por exemplo: o método **Processar_Salario()** pode ser invocado de forma diferente quando tratar-se de um **Funcionário Regular** (sem subsídios) ou **Funcionário Chefe** (com subsídio), mas todos eles sendo sub classes de **Funcionário**. Este princípio será discutido na unidade III no aula sobre a sobrecarga de métodos.

Este princípio consiste numa variante de abstracção e torna o código mais simples pois facilmente simula a hierarquia do mundo real.

O outro exemplo de polimorfismo consiste no operador “+”, que em alguns casos pode assumir a função de somatório, e em outros casos a função de concatenação:



No primeiro caso o sinal + executa a soma de a e b pois os dois são números, contudo, no segundo caso, o sinal + efectua a concatenação pois a e b são textos. Daí, o sinal + assume múltiplas formas.

2.6 Mensagem

Quando instanciamos um objecto, com o objectivo de chamar os seus métodos consideramos isto uma mensagem. Um objecto pode também invocar métodos de outros objectos, havendo assim uma troca de mensagens entre ambos.

Por exemplo, numa transferência bancária entre o objecto1 (cliente1) e o objecto2 (cliente 2), quando o valor é reduzido do saldo do cliente e acrescentado ao saldo do cliente2 estamos numa situação de troca de mensagem entre dois objectos.

Assim, terminamos a discussão sobre os princípios básicos das linguagens OOP. No próximo tópico iremos discutir alguns exemplos da aplicação das linguagens OOP bem como a sua importância:

3 – Importância das linguagens OOP

O paradigma OOP veio mudar a forma como os programas eram escritos, dando mais foco à organização em unidades chamadas de objectos. Esta abordagem traz várias vantagens:

3.1 Facilidade de modelagem

As linguagens OOP partem do princípio de que tudo na vida pode ser representado em classes e objectos. Deste modo, com o aparecimento de linguagens de modelação como a UML (Unified Modelling Language) tornou-se mais fácil representar o problema, pois este possui uma variedade de símbolos/diagramas que permitem transformar qualquer problema em modelo OOP.

3.2 Reutilização do código

Uma instrução repetitiva pode ser agregada em métodos que por sua vez serão agrupados em classes. Usando os princípios como herança ou polimorfismo é possível reutilizar um método reduzindo o número de linhas de código do software e reduzindo também o tempo de execução.

3.3 Facilidade de manutenção do software

A programação OOP aproxima-se muito à realidade humana, tornando fácil a sua leitura e correcção de erros. Este facto permite que não haja dependência do programador, pois qualquer programador pode facilmente fazer a manutenção.

3.4 Protecção

Na OOP os dados e métodos estão encapsulados em classes, permitindo que, apenas os membros desejados possam ser acedidos fora da classe, através dos diferentes tipos de visibilidade existentes (public, protected, friend, private).

3.5 Mais facilidades como:

- Objectos podem coexistir sem que haja interferência entre eles.
- Facilidade de actualização de pequenos para programas mais robustos
- O princípio de protecção de dados (private, public, protected) ajuda a melhorar a segurança da aplicação.

3.5 Aplicação da OOP

A programação orientada a objectos pode ser aplicada nas seguintes áreas:

- Simulação e modelação
- Inteligência artificial
- Websites dinâmicos
- Redes de computadores e em programação paralela
- Programação de dispositivos móveis



Actividade 1: Responda as seguintes questões

- 1.1 O que difere a programação orientada a objectos da programação estruturada?
- 1.2 Qual dos princípios OOP está relacionado à reutilização do código escrito?

Resposta

1.1 Na lição sobre os paradigmas de programação foi possível constatar que a programação orientada a objectos agrega as funções/procedimentos e variáveis em objectos e a programação estruturada transforma o problema em funções e procedimentos.

1.2 Nos princípios OOP vimos que a herança está ligada ao processo da reutilização do código.

[Fim da unidade I]

Por: Célio Barbosa Sengo

UNIDADE TEMÁTICA II Conceitos Básicos de programação (revisão)

Introdução geral

Esta unidade visa a introduzir a você a linguagem de programação orientada a objectos Java sob forma de revisão visto que os tópicos aqui discutidos fazem parte da Programação Imperativa por si aprendida no semestre passado (Variáveis, Estruturas de Controle, Funções, Matrizes e Manipulação de caracteres).

Assim, a presente unidade está dividida em X lições: Na lição 3 será discutido o conceito de variáveis e os principais tipos de dados, na lição 4 e 5 serão discutidos as principais estruturas de controle, na lição 6 serão discutidos de forma introdutória os conceitos Função, na lição 7 as Matrizes e na lição 8 a classe String.

Antes de atender a esta lição, vide nos materiais abaixo os passos para instalação e configuração do Java (JDK e JRE) no Windows.



Mais recursos

a) Instalação e Configuração do JDK

<https://www.youtube.com/watch?v=g9yxHSPN2PI>

Video com instruções básicas de como baixar, instalar e configurar o JDK no Windows 7

b) Instalando e Configurando o Java Development Kit 7 para Desenvolvimento

<http://luizricardo.org/2013/11/instalando-e-configurando-o-java-development-kit-7-para-desenvolvimento/>

Website contendo os procedimentos de configuração do JDK

LIÇÃO 3: VARIÁVEIS E TIPOS DE DADOS

1. Introdução

Após a discussão dos princípios de uma OOP na lição 2, segue-se nesta lição o primeiro contacto com a linguagem orientada a objectos, onde iremos escrever os nossos programas, discutindo as variáveis e tipos de dados, bem como entrada e saída de dados.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Escrever programas usando os principais tipos de dados da linguagem OOP
- Implementar a leitura (inserção) e saída (visualização) de dados na resolução de um problema.

2. Estrutura básica de um programa em Java

A programação básica em java consiste na técnica de fazer com que o computador esteja pensando através estruturas de controle que operam sobre variáveis. A seguir iremos discutir a estrutura do programa em java:

Documentação	}	Opcional
Declaração de pacotes		
Importação de pacotes		
Definição de Interfaces		
Definição de classes		
Definição da classe principal	}	Obrigatório

Sendo assim, um programa básico em java deve possuir o formato que se segue:

```

Class MinhaClasse {
    public static void main(String args[])
    {
        ....
    }
}

```

Onde **main** é onde serão invocados todos os métodos.

3. Variáveis e tipos de dados

Variável consiste em qualquer coisa capaz de armazenar um valor ou uma expressão e que expressa um endereço (posição) na memória.

Por exemplo, se quisermos armazenar a idade de alguém podemos criar uma variável idade. Mais do que dar o nome à variável, importa também definir o intervalo de valores possíveis que ela pode armazenar, bem como o seu tamanho na memória. É a isto que nós chamamos de tipo de dado. Existem regras básicas para declarar uma variável:

Variáveis	
Nunca devem iniciar com um valor numérico	idade, peso, x, y , ect
Maiúsculas e minúsculas são distintas	Idade é diferente de idade
Nunca devem ser iguais a palavras reservadas da linguagem de programação	nunca usar public como variável
Nunca devem ter espaçamentos	local de nascimento (errado) mas sim local_de_nascimento
O nome pode assumir qualquer tamanho	dados_pessoais

3.1 Os principais tipos de dados

A tabela a seguir ilustra os principais tipos de dados, seus respectivos tamanhos na memória e o intervalos de valores suportado.

Introdução à Programação Orientada a Objectos

Em java temos os seguintes tipos de dados:

Tipo	Descrição	Tamanho na memória	Intervalo de valores
byte	Números inteiros	1 byte	-128 a 127
short	Números inteiros	2 bytes	-32768 a 32767
int	Números inteiros	4 bytes	-2147483648 a 2147483648
long	Números inteiros	8 bytes	-2^{63} a 2^{62}
float	ponto flutuante	4 bytes	3.4e-038 a 1.7e038
double	ponto flutuante	8 bytes	3.4e-308 a 1.7e+308
string	sequência de caracteres	2 bytes (por caracter)	
classe	objectos	depende	
array	variável capaz de armazenar uma sequência de valores distintos (números, caracteres, objectos, etc)	depende do tipo	



Exemplo:

```
byte a;  
a=110;
```

ou ainda podemos, declarar e inicializar numa só linha.

```
byte a=110;
```



Atenção:

```
byte a=129; //dá erro pois foi violado o intervalo de valores possíveis
```

```
float x=1.2;
```

e a instrução:

```
int a=x; //dá erro pois uma variável inteira não pode receber um valor real
```

Seria necessário realizar uma conversão de um tipo de dado para o outro

3.2 Tipos de conversão

Como vimos no exemplo anterior, não podemos atribuir um valor a uma variável que possui um tipo de dado diferente do valor a ser atribuído. Para tal é necessário efectuar uma conversão de um tipo de dado para o outro:

Existem dois tipos de conversão:

- Conversão Implícita (realizada de forma automática)
- Conversão explícita (realizada de forma manual pelo programador)

3.2.1 Implícita

Quando temos variáveis tipo int, byte, long e short é possível atribuir o valor de uma variável para outra, desde que respeite o intervalo de números suportado pelo receptor. O mesmo acontece com variáveis double e float que são também compatíveis.



Considere o exemplo:

```
int x=127;  
byte b=x; //ouve aqui uma conversão automática
```

3.2.2 Explícita (Casting)

Usada para converter dados de um tipo de dado para o outro, mesmo que estes não sejam compatíveis.



Exemplo:

```
float f= 2.5;  
int x = (int) f;  
o x passa a ter o valor 2 pois a parte fraccionária é truncado
```

3.3 Entrada e saída de dados

Os programas são desenvolvidos para interagir com os utilizadores, onde devem introduzir seus dados pelo teclado.

Em java possui muitas classes que permitem interagir via teclado: o Scanner e o

DataInputStream são das mais usadas. A classe Scanner possui uma série de métodos para ler dados dependendo do tipo de dado:

- next() e nextLine() usado para ler textos
- nextInt() para ler números inteiros
- nextFloat() para ler números reais
- nextDouble() para ler números reais

É importante escolher o método que se adequa ao tipo de dado no qual pretendemos armazenar o valor digitado no teclado. Por exemplo, se for inteiro então escolhemos o nextInt().



Por exemplo:

Pretende-se desenvolver um programa onde o utilizador introduz dois números e o sistema calcula e imprime a soma dos dois.

```
class Mat
{
    public static void main(String args[])
    {   int X, Y;
        Scanner Teclado=new Scanner(System.in);
        System.out.print("Digite o primeiro numero");
        X=Teclado.nextInt();
        System.out.print("Digite o segundo numero");
        Y=Teclado.nextInt();
        int R=X+Y;
        System.out.print("A soma="+R);
    }
}
```



Atenção:

Para imprimir textos misturados com variáveis usa-se o operador + na impressão.
System.out.print("A soma="+R);

LIÇÃO 4: ESTRUTURAS DE CONTROLE [CONDICIONAIS]

1. Introdução

Na lição 3 foram discutidos os principais tipos de dados, permitindo assim criar programas onde podemos inserir e visualizar dados. Contudo, um programa não feito apenas de entrada e saída de dados, contendo também instruções que possibilitam desviar o fluxo do programa, dando a ideia de que o computador é inteligente. Por exemplo, quando introduzimos o pin do nosso telemóvel este consegue nos dizer se este está correcto ou não. Isto é possível através de tais desvio, recorrendo às diferentes instruções/estruturas de controle, como as condicionais, repetição e salto.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Compreender e aplicar as estruturas de controle condicionais (IF e SWITCH) na resolução de problemas em ciências de computação.

4.2. Estruturas de controle

As estruturas de controle são usadas para desviar o fluxo tradicional de um programa (topdown), fazendo com que o computador pareça estar tomando decisão. As estruturas de controle ajudam-nos também a processar de forma repetitiva uma série de valores.

A seguir vamos discutir as categorias existentes no Java:

Condicionais	Repetição	Salto
IF	DO	BREAK
SWITCH	WHILE	CONTINUE
	FOR	

4.2.1 Estrutura condicional IF

Usado juntamente com os operadores relacionais/lógicos para controlar a execução do programa. Se uma certa condição for verdadeira então um certo conjunto de instruções será executado.

Sintaxe:

```
if <condição>
{ conjunto de instruções }
```



Exemplo:

Um programa que verifica se um número é maior que Zero.

```
int a=10;
if(a>0)
{
    System.out.println("Maior que Zero");
}
```

Neste caso a condição $a > 0$ é verdadeira daí a instrução `System.out.println("Maior que Zero")` será executada pelo que o output será: Maior que Zero.

4.2.2 Variantes do "if"

Existem algumas variantes existentes no "if" e sua aplicação depende do problema em causa.

a) if...else (com duas vias/saídas)

Neste caso temos duas vias: a verdadeira e a falsa, assim podemos programar o que vai acontecer em cada uma das condições:

```
int a= - 10;
if(a>0)
```



```
{ System.out.println("Maior que Zero"); }  
else  
{ System.out.println("Menor que Zero"); }
```

Sendo assim, se a primeira condição for verdadeira então a sua instrução será executada, caso não, a instrução do else será executada. E como resultado teremos: "Menor que zero".

b) if...else if (em escada)

Usado para concatenar várias condições em escada, sendo que apenas a primeira condição verdadeira é será executada, as restantes (por mais que sejam verdadeiras) serão ignoradas.

Exemplo:

```
int a=10;  
if(a>0)  
{ System.out.println("Maior que Zero"); }  
else if(a = 10)  
{ System.out.println("Dez"); }  
else if(a<0)  
{ System.out.println("Menor que zero"); }
```

Neste caso, o $a > 0$ é verdadeiro daí apenas o `System.out.println("Maior que Zero");` será executado mesmo que exista outra condição verdadeira mais abaixo.

c) if...if...(composto)

Situações onde temos várias condições independentes uma da outra. Mesmo que uma delas seja verdadeira, as restantes serão testadas também.

Exemplo

```
int a=10;  
if(a>0)  
{ System.out.println("Maior que Zero"); }  
if(a = 10)  
    { System.out.println("Dez");  
  }  
if(a<0){ System.out.println("Menor que zero");  
}
```

```
else  
{ System.out.println("Positivo");}
```

Neste caso, o output será: Maior que Zero Dez Positivo pois cada um dos if foi testado individualmente.

4.2.3 Estrutura condicional switch

Existem situações onde o programa torna-se grande e complexo, devido a número elevado de condições que tornam difícil ler o código. Java possui uma forma de escrever tantos "ifs" de forma fácil: o switch. O switch é usado para testar a igualdade de valores inteiros/caracteres e são muito usados em menus.

Sintaxe:

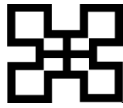
```
switch (variável)  
{  
    case valor 1:  
        conjunto de instruções  
        break  
    case valor 2:  
        conjunto de instruções  
        break;  
    ...  
    default:  
}
```

Exemplo:

Queremos testar se um certo número é par ou ímpar. Se o resto da divisão modular deste por dois for 0 então é par, caso não, é ímpar.

```
int x=4;  
int R=x%2;  
switch(R)  
{  
case 0:  
System.out.print("Par");  
break;  
case 1;  
System.out.print("Ímpar");  
break;  
default:  
System.out.print("Nao é par e nem ímpar...kakakaak");  
}
```

Neste caso é aconselhável colocar o break de modo que sejam ignoradas as instruções subsequentes após a execução do caso.



Actividade 2:

Escreva um programa onde o utilizador digita três notas dos seus testes e o sistema deve imprimir a média e dizer se está aprovado, reprovado ou dispensado, dependendo da média.

Resultado esperado:

```
import java.util.Scanner;
public class Media {
    public static void main(String[] args) {
        Scanner ent = new Scanner(System.in);
        int nota1, nota2, nota3;
        int media, i, contAluno = 0;
        for(i = 0; i < 3; i++){
            contAluno++;
            // recebe a 1º nota
            System.out.println("Aluno " + contAluno + ", digite sua 1ª nota");
            nota1 = ent.nextInt();
            // recebe a 2º nota
            System.out.println("Aluno " + contAluno + ", digite sua 2ª nota");
            nota2 = ent.nextInt();
            // recebe a 3º nota
            System.out.println("Aluno " + contAluno + ", digite sua 3ª nota");
            nota3 = ent.nextInt();
            // calcula a média
            media = (nota1 + nota2 + nota3) / 3;
            System.out.println("A média do aluno " + contAluno + " é " + media);
            // mostra a nota do aluno
            if( (media >= 0) && (media <4) ){
                System.out.println("Nota E");
            } else if(media < 5){
                System.out.println("Nota D");
            } else if(media < 7){
                System.out.println("Nota C");
            } else if(media < 8){
                System.out.println("Nota B");
            } else if(media <= 10){
                System.out.println("Nota A");
            }
        }
    }
}
```

```
}  
}  
}  
}
```

LIÇÃO 5: ESTRUTURAS DE CONTROLE [REPETIÇÃO]

5.1 Introdução

Na lição 4 foram discutidos as principais estruturas de controle com o foco na condição “IF/SWITCH” que permitem o computador tomar uma decisão através de uma condição verdadeira ou falsa. Por vezes o computador precisa realizar uma certa tarefa repetidas vezes ou processar grandes volumes de dados. Para tal estruturas de repetição são usadas. Por exemplo, contagem do número de aprovados em uma turma, carece deste tipo de estruturas. Assim, na presente lição iremos discutir as estruturas Do, While e For.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Compreender e aplicar as estruturas de repetição na resolução de problemas em ciências de computação.
- Aplicar as estruturas de salto na resolução de problemas

.

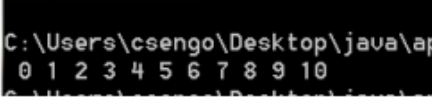
5.2 Estrutura de Repetição “Do e While loop”

Estas estruturas tem a notação: **faça ... enquanto a condição for verdadeira (Do)** ou **enquanto a condição for verdadeira faça ..**

WHILE	DO
<pre>while (condição verdadeira){ Conjunto de instruções }</pre>	<pre>do { conjunto de instruções } while (condição for verdadeira)</pre>



Suponhamos que pretendemos imprimir uma série de número de 0 a 10;

WHILE	DO
<pre>int i=0; while (i<=10){ System.out.print(i); i++; }</pre>	<pre>int i=0; do{ System.out.print(i); i++; }while (i<=10)</pre>
	
Enquanto o i for menor que 10: imprima i, incremente e teste.	Imprima i e incremente: enquanto o i for menor que 10.
O teste da condição é realizado logo no início do loop.	O primeiro loop é executado, só depois é testada a condição.

No exemplo acima o resultado é o mesmo tanto usando o while, como usando o do.



A seguir veremos um exemplo que traz resultados distintos:

WHILE	DO
<pre>int i=11; while (i<=10){ System.out.print(i); i++; }</pre>	<pre>int i=11; do{ System.out.print(i); i++; }while (i<=10)</pre>
	11
Não teremos nenhum resultado pois logo a prior a condição $11 \leq 10$ é falsa, daí o while não vai se quer iniciar.	O primeiro loop é executado, imprimindo 11, depois é que verifica-se que de facto a condição é falsa e o loop é interrompido.

Tanto o Do como o while são muito usados em programação, dependendo da situação em causa.

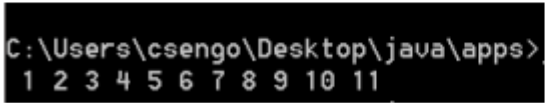
5.3 Estrutura de Repetição “FOR”

Quando sabe-se de antemão o número exacto de repetições desejadas recorremos a estrutura “for”.

Sintaxe:

FOR
for (valor inicial ; condição; incremento/decremento){ Conjunto de instruções }

Por exemplo, pretende-se de novo imprimir uma série de número de 1 a 11;

<pre>for (int i=1; i<=11; i++){ System.out.print(i); }</pre>
output: 

6.2 Estrutura de salto “BREAK” e “CONTINUE”

O break e continue são usados para alterar o ciclo de repetições. Usamos o break para interromper o ciclo e usamos o continue para saltar o ciclo corrente e passar para o próximo.

Break

Suponhamos que pretendemos que o utilizador introduza a senha correcta para entrar no sistema. Ele entra num ciclo onde só sai quando a senha correcta é introduzida.

Solução:

```
class ABC
{
public static void main(String args[])
{ Scanner T=new Scanner(System.in);
int i=0, senha;
do{
System.out.println("Digite a senha correcta por favor");
senha=T.nextInt();
if(senha==12345);
{
System.out.print("Senha Correcta, bem vndo ao sistema");
break;
}
} while (i<10)
}
```

Preste atenção que estamos num ciclo infinito, pois o valor de "i" nunca sofre alteração, isto é, permanece sempre menor que 10 (pois nunca chega a incrementar), daí, o ciclo só será interrompido caso condição senha==12345 através do break.

Continue

Suponha que pretende imprimir todos os números ímpares no intervalo de 1 a 100. Podemos recorrer ao continue para ignorar a impressão dos números pares. Recorde que números pares são aqueles cuja divisão modular por 2 é sempre zero.

```
C:\Users\csengo\Desktop\java\apps>javac Programa2.java
C:\Users\csengo\Desktop\java\apps>java Programa2
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49 51 53 55
57 59 61 63 65 67 69 71 73 75 77 79 81 83 85 87 89 91 93 95 97 99
C:\Users\csengo\Desktop\java\apps>_
```

Solução:

```
class ABC
{
public static void main(String args[])
{
for(int i=1; i<=100; i++)
{
```



```
if(i%2==0)
{
continue;
}
System.out.print(i);
}
}
}
```

Podemos aqui constatar que a instrução `System.out.print(i)` será ignorada caso o tal “i” seja par através do `continue`.

LIÇÃO 6: FUNÇÕES

6.1. Introdução

As funções/métodos desempenham um papel muito importante na programação, pois permitem criar instruções que realizam uma tarefa específica e invoca-las sempre que necessário, eliminando a possibilidade de reescrever a instrução cada vez que se pretende usa-la. Deste modo, na presente lição iremos discutir as funções, os seus tipos e a forma de invocar-las.

Objectivo geral da aula/Meta



Assim, no fim desta aula esperamos que você seja capaz de:

- Desenhar programas OOP usando funções/métodos
- Demonstrar a diferença entre os 2 tipos de funções/métodos

6.2 Funções/Métodos

Como foi discutido na lição 2 os métodos (muitas vezes chamados de “funções”) ditam o comportamento do objecto, através da alteração do conteúdo dos dados (variáveis). Os métodos são declarados dentro da classe, podendo estes ser invocados no main ou em outros métodos (um método que chama outro método), permitindo assim reutilizar a mesma instrução várias vezes.

Para declarar um método usamos a sintaxe a seguir:

```
Tipo nome (lista de parâmetros) {  
...corpo do método  
}
```

Onde,

Tipo: Tipo de retorno (void/não void)

Nome: Nome do método

Parâmetros: Lista de argumentos

Corpo: Instruções do método (variáveis, operadores, estruturas de controle, entre outros).



A seguir iremos criar uma função que recebe um valor em Dolar, converte e imprime o resultado da sua respectiva conversão para Metical usando a taxa cambial de 1USD = 60.0 MT.

Preste atenção aos comentários

a) Usando uma função void

```
class Programa{
    public static void Cambio (float Dolar) //declaracao da funcao
    {
        Float MT = Dolar/60; //conversao para MT
        System.out.println("Conversao para MT:"+MT); //impressão MT
    }
    public static void main(String args[]){
        Cambio(1000); //chamamos a funcao com argumento 1000 USD
    }
}
```

Neste caso, a função void apenas processa a conversão e imprime o resultado, isto é, não restorna o valor, mas sim imprime-o.

b) Usando uma função não void

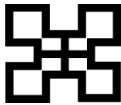
Uma função void retorna um valor específico correspondente ao tipo de dados na qual ela foi declarada. Neste caso o programa será tipo float;

```
class Programa{
    public static float Cambio (float Dolar) //declaracao da funcao
    {
        Float MT = Dolar/60; //conversao para MT
        Return MT; // armazenamento do MT na função Cambio
    }
    public static void main(String args[]){
        float R = Cambio(1000); //chamamos a funcao e armazenamos o
//conteúdo desta em R
        System.out.println("Resultado:"+R);
    }
}
```

A forma de invocar um método não void no main é diferente (do void) pois, este ;e visto como se fosse uma variável, sendo necessário apenas imprimi-la ou copiar o seu valor para outra variável.



Quando um método ou dado é declarado como estático significa que este pertence à classe e não aos objectos criado, podendo ser acedido no main sem a declaração do objecto. Mais detalhes sobre este conceito serão discutidos na unidade III (Classes, Objectos e Métodos).



Actividade

Crie um método que recebe notas de 2 testes, calcula a média e retorna o resultado dependendo da média (Aprovado ou reprovado). Chame este método no main.

Resposta

```
class ABC{
    public static String Verifica (float Nota1, float Nota2)
    {
        Float M = (Nota1 + Nota2)/2;
        if(M>10)
            return "Aprovado";
        else
            return "Reprovado";
    }
    public static void main(String args[]){
        float N1 = 14, N2 = 7;
        System.out.println("Resultado:"+Verifica(N1,N2));
    }
}
```

LIÇÃO 7: MATRIZES

7.1. Introdução

Na lição 3 discutimos que uma variável pode armazenar um valor, dependendo do seu tipo de dado. Contudo, há casos em que uma única variável pode ser usada para armazenar mais de um valor. A este tipo de variável chamamos de Matriz ou Array (inglês). Assim, nesta lição iremos discutir o conceito de matriz, a sua declaração, inicialização, bem como a matriz bidimensional e matriz de tamanho variável.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Preceber o processo de declaração e inicialização de uma matriz
- Compreender o conceito de matriz bidimensional
- Implementar matrizes na resolução de problemas

7.2 Matrizes/Arrays

Para melhor perceber o conceito, iremos recorrer a um exemplo. Vejamos a variável abaixo:

```
int Lista [ ] = new int [10];
```

Acabos de criar e inicializar uma matriz unidimensional “Lista” com 10 elementos inteiros.

Se quiséssemos criar uma matriz de elementos float faríamos:

```
float Lista [ ] = new float [10];
```

Características básicas de uma Matriz:

- Todos os elementos têm o mesmo tipo de dados
- Cada elemento é identificado por um índice que inicia de 0 até tamanho – 1
- Tem tamanho que corresponde ao número de elementos na lista

- Cada um dos elementos possui endereços seguidos (um seguindo o outro) de memória.

Lista[10]

0	1	2	3	4	5	6	7	8	9

Preenchimento

Após criar a lista de 10 elementos, o passo seguinte consiste em preencher cada um dos elementos.

Para tal só precisamos mencionar o índice tal como mostra o exemplo anterior:

```
float Lista [ ] = new float [10];  
Lista[0] = 16; // inserção do valor 16 na posição 0  
Lista [1] = 50; //inserção do valor 50 na posição 1
```

Contudo, preencher os elementos um a um, pode ser muito dispendioso quando a matriz tiver um tamanho maior, por exemplo 500. Para ultrapassar esta limitação usamos um loop, que inicia de 0 até tamanho – 1, neste caso de 0 a 9.

```
Scanner T = new Scanner (System.in);  
float Lista [ ] = new float [10];  
for(int i = 0; i < 10; i++)  
{  
    System.out.println("Digite o elemento"+i);  
    Lista[i] = T.nextFloat();  
}
```

Fazemos o mesmo quando pretendemos visualizar:

```
Scanner T = new Scanner (System.in);  
float Lista [ ] = new float [10];  
for(int i = 0; i < 10; i++)  
{  
    System.out.println("Elemento"+i+"="+Lista[i]);  
}
```

Programa completo:

```
class Prog{  
    public static void main(String args[])  
    {  
        Scanner T = new Scanner (System.in);  
        float Lista [ ] = new float [10];  
        for(int i = 0; i < 10; i++)
```

```

    {
        System.out.println("Digite o elemento"+i);
        Lista[i] = T.nextFloat();
    }
    for(int k = 0; k< 10; k++)
    {
        System.out.println("Elemento"+k+"="+Lista[k]);
    }
}
}

```

7.3 Matriz bidimensional

Existem casos em que se pretende criar uma lista com linhas e colunas (uma sequência de várias matrizes unidimensionais) recorrendo a matrizes. Por exemplo: uma lista de pagamentos feitos por 3 clientes onde cada cliente pode efectuar 3 pagamentos tal como mostra a figura abaixo:

ID	Pagamento1	Pagamento2	Pagamento3
1	100	250	500
2	200	120	150
3	300	350	500

Para tal criamos uma matriz bidimensional:

```
int Pag [][] = new int [ 3 ][ 4 ]
```

Esta matriz possui 3 linhas e 4 colunas, onde na primeira coluna iremos armazenar o ID do cliente e nas restantes colunas os pagamentos efectuados.

Assim para preencher a matriz iremos primeiro usar o método tradicional (um a um):

```
Pag[0][0] = 1; // Inserimos o valor 1 na linha 0 coluna 0
Pag[0][1] = 100; // Inserimos o valor 100 na linha 0 coluna 1
```

Tal como vimos no caso anterior, recomenda-se o uso de loops (ciclo de repetição) para inserir/visualizar a lista. Neste caso, iremos recorrer a dois loops, estando um dentro do outro, onde o loop exterior controla as linhas e o loop interno controla as colunas:

```

int Pag[ ][ ] = new int[ 3 ][ 4 ];
for(int i=0; i<3; i++) // loop das linhas
{
    for(int j=0; j<4; j++) //loop das colunas
    {
        System.out.println("Digite o elemento"+i + " " +j);
        Pag[ i ][ j ] = T.nextInt(); //inserção
    }
}

```

```
}
```

Para visualizar usamos o mesmo princípio (loop dentro de loop)

```
int Pag[ ][ ] = new int[ 3 ][ 4 ];
for(int i=0; i<3; i++) // loop das linhas
{
    for(int j=0; j<4; j++) //loop das colunas
    {
        System.out.println(Pag[ i ][ j ]);
    }
    System.outp.println(); //quebra de linha na visualização
}
```

Em Java é possível criar uma matriz multidimensional com tamanho variável, fazendo o seguinte:

```
int K [ ] [ ] = new int [ 3 ] [ ]; //loop das linhas é fixo, o das colunas é variável
```

Para inicializar é necessário inicializar cada uma das colunas em separado:

```
int K [ ] [ ] = new int [ 4 ] [ ];
K[0] = new int [ 3 ]; // primeira linha terá 3 colunas
K[1] = new int [ 2 ]; // primeira linha terá 2 colunas
K[2] = new int [ 4 ]; // primeira linha terá 4 colunas
```

Assim a matriz será:



Actividade 2

Programe uma aplicação que permite inserir uma lista de 20 pagamentos numa matriz (usando o teclado), e o sistema deve imprimir o somatório dos pagamentos efectuados.

Resposta

```
class ABC {
    public static void main(String args[])
    {
        Scanner T = new Scanner (System.in);
        int Lista [ ] = new int [20];
        int Soma = 0;
        for(int i = 0; i< 10; i++)
        {
```


Introdução à *Programação Orientada a Objectos*

```
System.out.println("Digite o pagamento"+i);
Lista[i] = T.nextInt();
Soma += Lista[i];
}
System.out.println("Somatorio"+Soma);
}
}
```

LIÇÃO 8: MANIPULAÇÃO DE CARACTERES (STRING)

8.1. Introdução

Até agora vínhamos tratando as matrizes como uma lista de números (inteiros ou reais). Contudo, uma String pode ser vista como uma matriz unidimensional composta por uma sequência de caracteres. Assim, na presente lição, iremos discutir como manipular uma String através dos diferentes métodos existentes. Por exemplo, podemos concatenar dois nomes, ou converter a primeira letra para maiúscula, entre outras operações.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Perceber os principais métodos de manipulação de letras em uma String
- Manipular as letras de uma String de modo a resolver diferentes problemas ligados a textos.

8.2 A classe String

A manipulação de textos é um dos elementos fortes da linguagem Java. A forma mais simples de criar uma matriz de elementos tipo char como ilustra o exemplo a seguir:

```
char Nome [ ] = new char [ 4 ];  
Nome [0] = 'J'; Nome [1] = 'A'; Nome [2] = 'A'; Nome [3] = 'A';
```

Contudo, a instrução fica mais simples quando usamos o String:

```
String Nome = "JAVA"; //método 1  
String Apelido = new String ("Linguagem"); //método 2
```

As Strings podem ser facilmente concatenadas usando o operador de adição (+).

```
String NomeCompleto = Nome + Apelido;  
System.out.println(NomeCompleto);
```

Podemos também criar uma matriz de Strings, fazendo o seguinte:

```
String Lista [ ] = new String [ 4 ];
```

Onde temos uma lista capaz de armazenar 4 nomes.

8.3 Métodos da classe String

Um dado tipo String contém uma variedade de métodos que ajudam a manipular facilmente uma String.

A seguir iremos listar alguns métodos e demonstrar como usa-los:

Consideremos a String S;

Método	Utilidade
String S2 = S.toUpperCase();	Converte uma String para maiúscula
String S3 = S.toLowerCase();	Converte uma String para minúscula
S.equals(S2);	Compara duas Strings e retorna o resultado (true/false)
S.length();	Retorna o Tamanho da String
S.charAt(n);	Retorna a letra presente na posição n
String X = S1.concat(S2);	Concatena duas Strings
String W = S1.substring(n,m);	Retorna as letras no intervalo m e n



Mais métodos

Mais métodos podem ser encontrados ao executar o comando seguinte (no command prompt):

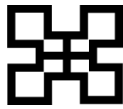
```
> javap java.Lang.String
```



Exemplo Completo

```
class Textos
{
    public static void main(String args[])
    {
        String X = "Maputo";
        System.out.println("Tamanho : "+x.length());
        System.out.println("Primeiro caracter : "+x.charAt(0));
        String k = x.toUpperCase();
        System.out.println("Convertido a Maiusculo :"+k);
        System.out.println("K==x? "+x.equals(k));
    }
}
```

}



Auto Avaliação

Parte 1: Marque se cada uma das afirmações é verdadeira (V) ou falsa (F).

- a) O nome da classe Java deve ser o mesmo com o nome do ficheiro.
- b) Numa matriz unidimensional com 5 elementos dá erro se tentarmos aceder ao elemento na posição 5.
- c) Uma variável pode ter um nome como **1valor**.

Parte 2: Marque a(s) alternativa(s) correcta(s) dentre as seguintes afirmações:

2.1 O método charAt() pertence à classe

- a) Integer b) Scanner c) String d) main

2.2 O resultado da expressão:

int a=5, b=2;

float R = a/b;

System.out.println(R);

Será:

- a) 2 b) 2.5 c) 0 d) erro

Respostas

Parte 1:

- a) A afirmação é verdadeira porque de facto a classe deve ter o mesmo nome com o ficheiro.
- b) Verdadeiro, pois, Dá erro sim, pois o índice da matriz inicia de 0 a Tamanho -1, isto é, de 0 a 4.
- c) Falso: O nome de uma variável não deve iniciar com um valor numérico.

Parte 2:

2.1 A alternativa correcta é a **c)**

2.2 A alternativa correcta é a **d)**

[FIM DA UNIDADE II]

UNIDADE TEMÁTICA III Classes, objectos e métodos

Introdução geral

Na unidade I, II e III foram discutidos os conceitos fundamentais de programação de forma generalizada, isto é, sem recorrer muito aos princípios OOP. Deste modo, nesta unidade pretendemos aplicar os conhecimentos discutidos usando os princípios OOP, onde serão detalhados os princípios discutidos na lição 2 (Classes, Objectos, Herança, Polimorfismo), combinando-os os conceitos discutidos na unidade II (estruturas de controle, funções, matrizes e Strings).

Assim, a presente unidade está dividida em 7 lições, onde na lição 01 serão discutidas as diferentes técnicas de manipulação de classes e objectos, na lição 2 será discutido o conceito de construtor, na lição 3 o conceito “static”, na lição 4 será abordado o princípio da sobrecarga (polimorfismo), na lição 5 serão tratados os objectos como argumentos de métodos, na lição 6 a matriz de objectos e na lição 7 serão abordados os pacotes.

LIÇÃO 1: DADOS E INSTÂNCIAS DE UMA CLASSE

1.1. Introdução

Na unidade I foram apresentados os princípios de uma linguagem OOP, onde vimos que um objecto é uma instância de uma classe, sendo que as instâncias de uma classe partilham todas as mesmas características. Nesta lição iremos discutir os procedimentos técnicos de como criar uma classe e as suas respectivas instâncias.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Compreender a estrutura de uma classe
- Criar uma classe com os seus respectivos dados
- Instanciar uma classe criada de modo inicializar e visualizar os seus dados

1.2 Estrutura e criação de uma classe

Uma classe em Java possui a seguinte estrutura:

```
class nome da classe [extends]
{
  [definição de dados]
  [definição de métodos]
}
```

Há que lembrar que uma classe contém métodos e dados e o método main é obrigatório na classe principal. Daí, em programação orientada a objectos as variáveis são designadas de “dados”.



VARIAVEIS = DADOS

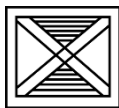


Exemplo: Pretende-se criar a seguinte classe

Funcionario

ID:
Nome:
Idade:
Salario:
VerDados()
AumentarSalario()

O funcionário em causa possui como dados o ID, Nome, Idade e Salário, e contém os seguintes métodos: VerdadDos() que irá imprimir os dados e AumentarSalario() que terá a função de incrementar o valor do dado “salario”.



“Os métodos da classe podem aceder e manipular directamente os dados de modo a alterar o comportamento do objecto”

Passo 1: Definimos os dados (variáveis e seus tipos de dados)

```
class Funcionario
{
    public int ID;
    public String Nome;
    public int Idade;
    public int Salario;
}
```

Passo 2: Criar o método principal, o main(), que deve chamar os dados e métodos da classe:

```
class Funcionario
{
    public int ID;
    public String Nome;
```

```
public int Idade;  
public int Salario;  
public static void main(String args[])  
{  
}  
}
```

Passo 3: Criamos os objectos do Funcionário. Por exemplo, podemos criar dois objectos, o F1 e F2.

```
class Funcionario  
{  
    public int ID;  
    public String Nome;  
    public int Idade;  
    public int Salario;  
    public static void main(String args[])  
    {  
        Funcionario F1=new Funcionario();  
        Fncionario F2=new Funcionario();  
    }  
}
```

Passo 5: Há que recordar que cada um dos objectos terá seus próprios dados e métodos e para aceder aos dados e métodos usamos o operador ponto “.”.

No exemplo a seguir, para introduzir os dados de F1. Por exemplo, usamos a instrução:

F1.ID=100 para introduzir o valor 100 ao ID do ID

A seguir o programa completo:

```
class Funcionario  
{  
    public int ID;  
    public String Nome;  
    public int Idade;  
    public int Salario;  
    public static void main(String args[])
```



```
{  
    Funcionario F1=new Funcionario();  
    Fncionario F2=new Funcionario();  
    F1.ID = 100;  
    F1.Nome="Joao";  
    F1.Salario=1200;  
    F2.ID=101;  
    F2.Nome="Joana";  
    F2.Idade=25;  
    System.out.println(F1.Nome);  
    System.out.println(F2.Nome);  
}  
}
```

Neste caso, estamos imprimindo os nomes de F1 e F2 através dos seus objectos.

Na próxima lição iremos discutir os métodos, tendo como base o exemplo aqui discutido.

LIÇÃO 2: MÉTODOS DE UMA CLASSE

2.1. Introdução

Na lição anterior discutimos os passos para criar uma classe, suas instâncias, onde foi criada a classe `Funcionário` e foram instanciados 2 objectos `F1` e `F2`. Os objectos criados só possuíam um único método, o `main`, contudo, é possível criar um objecto contendo mais métodos. Assim na presente lição iremos dar vida aos métodos de uma classe, adicionando mais métodos.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Adicionar métodos em uma classe de modo a manipular os dados
- Combinar os métodos e dados em uma classe
- Resolver problemas pontuais usando métodos em classes

2.2 Métodos

Métodos são um conjunto de instruções com a finalidade de resolver uma certa tarefa. Por exemplo, se temos instruções que simulam uma calculadora, podemos criar um método e chama-lo: `calculadora()`. Assim, sempre que pretendemos calcular a soma de dois números não precisamos repetir o código, mas sim, chamar o método. Os métodos podem ser invocados diversas vezes no programa.

Exemplo: Pretendemos criar um método que imprime “Hello World!!!”

```
class ABC
{
    public void ola()
    {
        System.out.print("Hello World!!!");
    }
}
```

}

Como chamar o método no main: Usamos a mesma notação que foi usada para os dados, através do operador “.”

```
class ABC
{
    public void ola()
    {
        System.out.print("Hello World!!");
    }
    public static void main(String args[])
    {
        ABC obj = new ABC();
        obj.ola();
    }
}.

```

Assim, o resultado do programa será de facto: **“Hello World”**

Algumas considerações

- Métodos são subprogramas que onde podemos escrever qualquer instrução
- As variáveis declaradas dentro dum método, pertencem ao tal método apenas.
- Um método pode invocar outro

2.3 Parâmetros/Argumentos

Como foi dito anteriormente as variáveis definidas dentro dum método pertencem ao tal método apenas. Quando pretende-se que um método receba dados do exterior usamos argumentos.

Se considerarmos um método “calculadora”, podemos fazer com que o método receba dois números do exterior e calcule a soma dos dois.

```
public void calculadora(int a, int b)
{
    int R = a + b;
    System.out.print("O Resultado="+R);
}
```

O valor de “a” e “b” serão definidos no momento em que o método for invocado.

Vejamos o programa completo:

```
import java.util.Scanner;
class Prog
{
    public void calculadora(int a, int b)
    {
        int R = a + b;
        System.out.print("O Resultado="+R);
    }
    public static void main(String args[])
    { int X,Y;
      Prog obj = new Prog();
      Scanner T = new Scanner(System.in);
      System.out.print("Digite o primeiro nr");
      X = T.nextInt();
      System.out.print("Digite o segundo nr");
      Y = T.nextInt();
      obj.calculadora(X,Y);
    }
}
```

Neste exemplo, o valor de X será copiado para a variável “a” e o valor de Y será copiado para o argumento “b”. Deste modo, o método calculadora imprime a soma

de dois números quaisquer “a” e “b” que serão conhecidos no main, na invocação do método em causa.



Se uma função têm dois argumentos inteiros, é obrigatório fazer referência aos tais argumentos no momento em que estamos invocando o método:

obj.calculadora(); vai dar erro pois a calculadora tem dois argumento.

2.4 Combinação de métodos e dados em classes

Como foi discutido anteriormente as classes possuem métodos e dados e a finalidade dos métodos em OOP é manipular os dados da classe.



Sendo assim, “Todos os métodos da classe têm acesso directo aos dados/métodos da classe a que eles pertencem”.

Retomemos ao caso concreto do programa da lição anterior:

Funcionario

ID:
Nome:
Idade:
Salario:
VerDados()
InserirDados()
AumentarSalario()

O método “VerDados” deve imprimir os dados do funcionário, o método InserirDados será usado para introduzir dados.

A seguir iremos completar o programa, adicionando instruções nos dois métodos e por conseguinte, invoca-los no main:

```

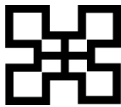
class Funcionario
{
    public int ID;
    public String Nome;
    public int Idade;
    public double Salario;
    public void VerDados()
    {
        System.out.print("Nome = "+Nome);
        System.out.print("Idade = "+Idade);
        System.out.print("Salário = "+Salario);
    }
    public void InserirDados(String x, int y, int z)
    { Nome = x; Idade = y; Salario = z; }
    public static void main(String args[])
    { Funcionario F1=new Funcionario();
      F1.InserirDados("Manuel", 35, 2400.0);
      F1.VerDados();
      System.out.println("-----");
      Funcionario F2=new Funcionario();
      F2.InserirDados("Joana", 28, 3400);
      F2.VerDados();
    }
}

```

Como pode ver, foram usados os métodos para inserir e visualizar os dados dos funcionários F1 e F2 recorrendo aos seus métodos, sendo esta uma melhoria em relação à solução apresentada na lição anterior. O método InserirDados(int x, int y, int z) recebe três parâmetros que alimentam os dados do funcionário: x vai alimentar o nome, y vai para a idade e z vai para o salário. O método VerDados acede ao nome, idade e salário do objecto e imprime.

Ao executar o programa teremos o seguinte resultado:

```
C:\Users\csengo\Desktop
Nome = Manuel
Idade = 35
Salario = 2400
-----
Nome = Joana
Idade = 28
Salario = 3400
```



Actividade 1: Considere a seguinte classe:

Devedor
String ID: String Nome: Float Dívida: Float Pagamento
void VerDados() void InserirDados(String, String, float, float) void Estado()

- a) Complete os métodos da classe de modo que:
- i) VerDados visualize todos os dados do Devedor
 - ii) InserirDados permite inicializar o ID, o Nome e a Dívida do devedor
 - iii) Estado verifica se o devedor está em dívida ou não.
- b) Crie dois objectos (devedores) e imprima a soma dos pagamentos e das dívidas dos dois.

Resposta

```
class Devedor {  
    public String id;  
    public String nome;  
    public float divida;  
    public float pagamento;
```

```
//Inicialização dos dados
public void verDados(){
    System.out.println("ID:"+this.id+"Nome:"+this.nome+"\n");

    System.out.println("Pagamentos:"+this.pagamento+"Divida:"+this.divida+"\n");
}

//Inserção de dados
public void inserirDados(String ID,String NOME,float DIVIDA,float PAGMENTO)
{ this.id=ID; this.nome=NOME;this.divida=DIVIDA; this.pagamento=PAGMENTO;}

//Verificação do estado.
public void estado()
{ float saldo = pagamento-divida;
  if(saldo>=0)
      System.out.print(this.nome+":Sem dívida, com saldo"+saldo);
    else
      System.out.print(this.nome+":Devedor, com saldo"+saldo);
}

public static void main(String args[]){
    //criação dos objectos
    Devedor d1 = new Devedor();
    Devedor d2 = new Devedor();

    //inserção e visualizacao
    d1.inserirDados("D1","Abel",1200,200);
    d1.verDados();
    d2.inserirDados("D1","Joelma",2000,2400);
    d2.verDados();

    //Visualizacao do saldo
    d1.estado();
```



```
        d2.estado();  
    }  
}
```

O Resultado será:

```
ID:D1Nome:Abel  
Pagamentos:200.0Divida:1200.0  
ID:D1Nome:Abel  
Pagamentos:2400.0Divida:2000.0  
Abel:Devedor, com saldo-1000.0Joelma:Sem dívida, com saldo400.0
```

LIÇÃO 2: CONSTRUTORES

2.1. Introdução

Como foi discutido anteriormente, todos os dados dum objecto precisam ser inicializados antes de ser usados, isto é, devem ter um valor inicial. O método `InserirDados` foi usado anteriormente para inicializar os dados da classe, contudo, este processo pode ser realizado de forma automática através de Construtores. Assim a presente lição irá discutir os procedimentos de criação de construtores em classes.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Compreender o princípio básico de construtores em OOP
- Perceber as regras básicas da criação de construtores

2.2 Construtores

Considera-se construtor é um método que tem o mesmo nome com o da classe e será implicitamente executado imediatamente após a criação do objecto.

Para criar um construtor é ter em consideração as seguintes regras:

- Não deve ser void e nem não void (não tem tipo de retorno não)
- Tem o mesmo nome com o da classe
- É um método que pode ter argumentos como não



Se por exemplo temos a classe `Funcionário` podemos criar um construtor: `public Funcionário` que inicializa os dados funcionários, isto é, todo o funcionário que for criado terá por defeito os dados definidos no construtor.

```
class Funcionario
{
    public int ID;
    public String Nome;
    public int Idade;
    public double Salario;
    public void VerDados()
    {
        System.out.print("Nome = "+Nome);
        System.out.print("Idade = "+Idade);
        System.out.print("Salário = "+Salario);
    }
    public Funcionario( )
    {
        Nome = "xxx";
        Idade = 0;
        Salario = 1000;
    }
    public static void main(String args[])
    {
        Funcionario F1=new Funcionario(); //Construtor invocado
        Funcionario F2=new Funcionario();//Construtor invocado
        F1.VerDados(); F2.VerDados();
    }
}
```

No programa acima descrito, o construtor é invocado logo durante a criação do objecto através da instrução **new Funcionario()**.

Pudemos assim constatar que tanto o F1 como F2 terão por defeito terão Nome=xxx, idade= 0 e Salario=1000. Ai podemos alterar os dados individuais de cada um.

Introdução à *Programação Orientada a Objectos*

O construtor pode também possuir parâmetros, o que será discutido na próxima lição.

LIÇÃO 3: MÉTODOS E DADOS ESTÁTICOS

3.1. Introdução

Nos capítulos anteriores vimos que ao criar um objecto, este passa a conter todos os dados e métodos da classe. Contudo existem situações em que pretendemos criar um método ou dado que seja comum a todos objectos da classe de modo que estes possam partilha-lo. Isto só é possível se declararmos um método ou dado como **static**. Deste modo, nesta lição iremos apresentar o conceito “static” na programação OOP.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Entender o uso dos métodos e dados estáticos em OOP
- Implementar os métodos estáticos da classe Math em operações matemáticas.

3.1 Membros estaticos

Um método ou dado declarado como estático passa a pertencer à classe onde este foi declarado, podendo ser executado directamente através da classe, mesmo sem que exista algum aobjecto criado. Contudo, se forem criados objectos, o tal método estático passa a ser partilhado pelos objectos (diferente de outros dados/métodos não estáticos que pertencem a cada um dos objectos). Assim, o Java cria apenas uma cópia do membro estático.

REGRAS BÁSICAS

- Um membro estático pertence à classe e não ao objecto
- Um dado/método estático pode ser executado mesmo que não exista nenhum objecto criado, basta usar o nome da classe
- Um dado estático tem o valor zero por defeito
- Um método estático só pode aceder a dados/métodos estáticos



O exemplo a seguir pretende criar um programa onde temos vários objectos a efectuarem uma certa contribuição, depositando um valor numa variável chamada `doacao` que por defeito inicia com o valor zero.

Cada objecto deverá efectuar o incremento por 100 da variável `doacao` e no fim, usaremos o nome da classe para imprimir o valor existente na doação.

```
class Morador
{
    public static int doacao;
    public String Nome;
    public Morador( String n)
    {
        Nome = n;
    }
    public static void main(String args[])
    {
        Morador M1=new Morador("Carlos");
        Morador M2=new Morador("Joao");
        M1.doacao=M1.doacao+100;
        M2.doacao=M2.doacao+100;
        System.out.println("Valor total"+Morador.doacao);
    }
}
```

Como resultado teremos o seguinte resultado:

```
C:\Users\csengo\Desktop\java\apps>java Morador
Valor total: 200
```

Portanto, o M1 e o M2 incrementam a mesma variável por ela ser estática.

3.2 A Classe Math

O java possui a classe `Math` que possui uma série de métodos estáticos que realizam uma série de cálculos científicos como:

```
double sin(double);
public static double cos(double x); Seno
public static double tan(double x); Coseno
public static double log10(double x); Logaritmo na base 10
```

```
public static double sqrt(double x); Raíz quadrada  
public static double pow(double x, double y); x elevado a y
```



Para mais detalhes da classe Math execute o comando a seguir no command prompt:

javap java.lang.Math

Para aceder a este métodos basta chamar o método juntamente com o nome da classe como mostra o exemplo a seguir:



```
class Calc  
{  
    public static double Expoente(double a, double b)  
    {  
        return Math.pow(a,b);  
    }  
    public static void main(String args[])  
    {  
        double x=3,y=2;  
        System.out.println("Resultado: Usando Metodo Expoente  
"+Calc.Expoente(x,y));  
        System.out.println("Resultado: Chamando Math  
Directamente"+Math.pow(x,y));  
    }  
}
```

O resultado do programa acima será:

```
C:\Users\csengo\Desktop\java\apps>javac Calc.java  
C:\Users\csengo\Desktop\java\apps>java Calc  
Resultado: Usando Metodo Expoente 9.0  
Resultado: Chamando Math Directamente9.0
```

O método **pow()** da classe **Math** foi invocado dentro do método **Expoente** criado para retornar o expoente de **a** elevado a **b**. Outra estratégia é usar o **Math** directamente dentro do **main()** para realizar uma determinada operação. Tanto um método como outro é válido.



ATENÇÃO:

A classe **Math** pertence ao pacote **lan**, contudo, não é necessário fazer o **import** do **destepacote** na classe.

LIÇÃO 4: SOBRECARGA

4.1. Introdução

Em OOP é possível existir dois ou mais métodos com o mesmo nome na classe, desde que possuam parâmetros diferentes (número ou tipo de dados). Esta técnica é usada quando pretendemos que um ou mais objectos executem tarefas semelhantes através da entrada de dados (parâmetros) diferentes. Neste caso, é durante a execução do programa que o Java decide qual dos métodos irá executar, analisando os parâmetros usados no momento em que o método foi invocado no main. Este conceito é chamado de polimorfismo em OOP.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Conceber programar usando sobrecarga de métodos
- Fazer o uso de sobrecarga em construtores em programas OOP

4.2 Sobrecarga de métodos

Suponhamos que pretendemos criar dois métodos com o mesmo nome na classe, onde, onde o primeiro método `Area()` recebe dois parâmetros e calcula a área de um rectângulo e o segundo método também com o mesmo nome recebe um parâmetro e calcula a área de uma circunferência. Usando a sobrecarga podemos facilmente definir os dois métodos da seguinte forma:

```
void Area(float comp, float larg){
```

```
...//instruções para o cálculo da área do rectângulo  
}
```

E para a área do triângulo teríamos:

```
void Area(float raio){  
  
...//instruções para o cálculo da área do círculo  
}
```

Assim, durante a invocação do método o programa poderia escolher o método de acordo com o número de parâmetros usados no main.

Veja a seguir o programa completo:

```
class Prog{  
    void Area(float comp, float larg){  
        float R = comp*larg;  
        System.out.println("Area do rectangulo="+R);  
    }  
    void Area(float raio){  
        float R = 4.14 * (raio * raio);  
        System.out.println("Area do circulo="+R);  
    }  
    public static void main(String args[])  
    {  
        float c = 10, L = 2, r=10;  
        Prog obj = new Prog;  
        obj.Area(r); //Imprime a área do circulo  
        obj.Area(c,L); // imprime a área do rectângulo  
    }  
}
```

Deste modo, só durante a execução é que se sabe qual dos métodos será invocado.

4.3 Sobrecarga de construtores

Java permite que tenhamos na mesma classe dois ou mais construtores, desde que tenham argumentos diferentes (em número ou tipo de dados). Este conceito é chamado de sobrecarga de construtores.

No exemplo a seguir iremos demonstrar como efectuar a sobrecarga de construtores:

```
import java.util.Scanner;

class Funcionario
{
    public int ID;
    public String Nome;
    public int Idade;
    public double Salario;
    public Funcionario( )
    { Nome = "xxx"; Idade = 0; Salario = 1000; }
    public Funcionario(String n, int i, int s)
    { Nome = n; Idade = i; Salario = s; }
    public void VerDados()
    {
        System.out.print("Nome = "+Nome);
        System.out.print("Idade = "+Idade);
        System.out.println("Salario = "+Salario);
    }
    public static void main(String args[])
    { Funcionario F1=new Funcionario(); F1.VerDados();
      Funcionario F2=new Funcionario("Ana",38,3500); F2.VerDados();
```

```

    }
}

```

Como resultado teremos:

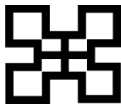
```

C:\Users\csengo\Desktop\java\apps>java Funcionario
Nome = xxxIdade = 0Salario = 1000.0
Nome = AnaIdade = 38Salario = 3500.0

```

Qual dos construtores será invocado?

O F1 foi inicializado com o construtor **new Funcionario()** sem argumentos, que vai chamar o construtor **public Funcionario** e o F2 foi inicializado com o **new Funcionario("Ana",38,3500)** que tem três argumentos (String, int, int) e que irá chamar o construtor com a mesma estrutura, neste caso o **public Funcionario ((String n, int i, int s)**. Seria um erro criar um objecto com um construtor com dois argumentos, pois não existe nenhuma sobrecarga com tal configuração na classe.



Actividade 2: Considere a seguinte classe (Da actividade anterior):

Devedor
String ID:
String Nome:
Float Dívida:
Float Pagamento
void VerDados()
void InserirDados(String, String, float, float)
void Estado()

a) Crie um construtor com parâmetros de modo que este inicialize os dados do devedor, tal como o método `inserirDados()` faz.

b) Crie um dado estático que permite que os devedores com saldo positivo possam doar os seus pagamentos neste dado. Após a doação, imprima o estágio de todos Devedores.

Resultado

```
class Devedor2 {  
    public String id;  
    public String nome;  
    public float divida;  
    public float pagamento;  
    public static float doacao;  
    //Inicialização dos dados  
    public void verDados(){  
        System.out.println("ID:"+this.id+"Nome:"+this.nome+"\n");  
  
        System.out.println("Pagamentos:"+this.pagamento+"Divida:"+this.divida+"\n");  
    }  
    //Construtor  
    public Devedor2(String ID,String NOME,float DIVIDA,float PAGMENTO)  
    { this.id=ID; this.nome=NOME;this.divida=DIVIDA; this.pagamento=PAGMENTO;}  
  
    //Verificação do estado.  
    public void estado()  
    { float saldo = pagamento-divida;  
      if(saldo>=0)  
          System.out.print(this.nome+":Sem divida, com saldo"+saldo);  
      else  
          System.out.print(this.nome+":Devedor, com saldo"+saldo);  
    }  
  
    //método para doacao: verificar o saldo, se for positivo transferir o valor a doar  
    //do pagamento para a variável estática  
    public void doar(float v){  
        float saldo = pagamento-divida;  
        if(saldo>=0)
```

```

        {
            this.pagamento-=v;
            doacao+=v;
        }
        else
            System.out.println(this.nome+"Nao tem saldo suficiente!!!");
    }

    public static void main(String args[]){
        Devedor2 d1 = new Devedor2("D1","Abel",1200,200);
        Devedor2 d2 = new Devedor2("D2","Joelma",2000,2400);
        d1.verDados();
        d2.verDados();
        d1.estado();
        d2.estado();
        System.out.println("-----Doando 50-----");
        d2.doar(50);
        d1.doar(50);
        System.out.println("-----Apos doacao-----");
        d1.verDados();
        d2.verDados();
        System.out.println("Valor existente na doacao:"+Devedor2.doacao);
    }
}

```

Resposta

E:\java Devedor2

ID:D1Nome:Abel

Pagamentos:200.0Divida:1200.0

ID:D2Nome:Joelma

Pagamentos:2400.0Divida:2000.0

Abel:Devedor, com saldo-1000.0Joelma:Sem divida, com saldo400.0-----Doan

do 50-----

AbelNao tem saldo suficiente!!!

-----Apos doacao-----

ID:D1Nome:Abel

Pagamentos:200.0Divida:1200.0

ID:D2Nome:Joelma

Pagamentos:2350.0Divida:2000.0

Valor existente na doacao:50.0

LIÇÃO 5: OBJECTOS COMO ARGUMENTO

5.1. Introdução

Na lição 10 vimos que um método pode possuir argumentos (parâmetros), permitindo que este receba dados do exterior durante a sua execução. Tais parâmetros podem ser de diferente tipo de dados (float, int, char, etc), contudo, os argumentos também podem receber variáveis do tipo objecto, isto é, argumentos objectos.

Assim, na presente lição, iremos discutir como criar um método que receba objectos como parâmetro.

Objectivo geral da aula/Meta



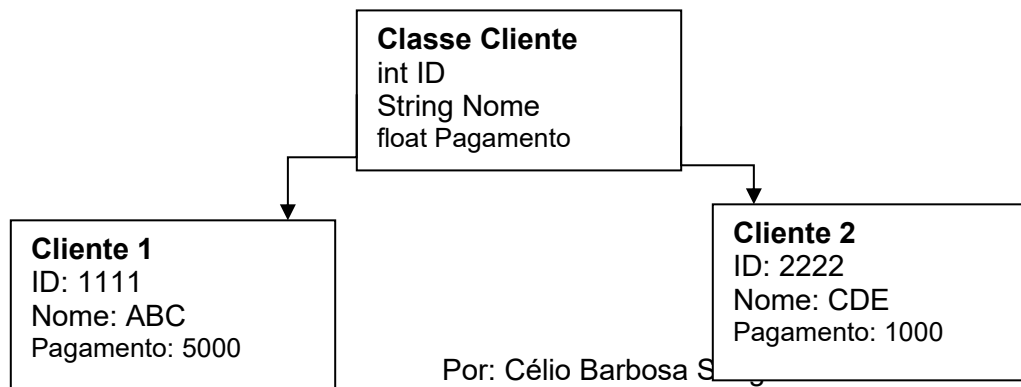
Assim, no fim desta aula o aluno deve ser capaz de:

- Compreender as formas de adicionar objectos nos argumentos de um método
- Aplicar métodos que recebam objectos como argumento no desenvolvimento de software.

5.2 Argumentos Objectos

Um método pode receber parâmetros do tipo objectos de modo que possa tratar dos membros de tais objectos.

Considere uma situação onde pretende-se criar uma classe Cliente que possua um método que permite retornar a soma dos pagamentos efectuados por 2 clientes tal como mostra a figura abaixo:



A soma dos pagamentos dos dois Clientes é 6000.

Podemos criar um método void Soma() que possui dois argumentos tipo Cliente e imprime a soma dos pagamentos de cada um dos dois objectos:

```
float Soma (Cliente C1, Cliente C2){  
    float Resp = C1.Pagamento + C2.pagamento;  
}
```

Assim, a variável Resp irá receber o valor da soma do pagamento do objecto C1 com o do objecto C2. Por conseguinte, para chamar este método no main() é necessário criar dois objectos que irão servir como argumentos deste.



Quando um método possui argumentos, estes devem ser indicados no momento em que o método é invocado.

Assim o main teria a seguinte codificação:

```
Cliente C1, C2, obj;  
C1 = new Client(); C2=new Cliente(); obj=new Cliente();  
C1.ID=1111; C1.Nome="ABC";  
C2.ID=2222; C2.Nome="CDE";  
System.out.println("A Soma="+obj.Soma(C1,C2));
```

A seguir o programa completo:

```
class Cliente{  
    int ID; String Nome; float Pagamento;  
    float Soma (Cliente C1, Cliente C2){  
        float Resp = C1.Pagamento + C2.pagamento;  
    }  
}
```

```
public static void main(String args[]){  
    Cliente C1, C2, obj;  
    C1 = new Client(); C2=new Cliente(); obj=new Cliente();  
    C1.ID=1111; C1.Nome="ABC";  
    C2.ID=2222; C2.Nome="CDE";  
    System.out.println("A Soma="+obj.Soma(C1,C2));  
}  
}
```

Deste modo, foi possível criar um método que recebe objectos como argumentos.

LIÇÃO 6: MATRIZ (ARRAY) DE OBJECTOS

6.1. Introdução

Na lição anterior discutimos as técnicas de como criar um método que recebe objectos como parâmetros, onde criamos dois objectos e na lição 7 vimos que uma matriz era uma variável capaz de armazenar mais de um único valor, sendo esta considerada uma matriz de valores. Os objectos também podem ser matrizes, isto é, podemos criar uma matriz de objectos de modo a tornar o programa mais simples. Imagine que pretendemos criar e inicializar 100 objectos da classe Cliente do programa anterior, não seria conveniente criar 100 variáveis uma a uma. E se fossem 1000? A solução passa por criar uma matriz de objectos.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Criar uma matriz de objectos capaz de armazenar vários dados
- Conceber programas simplificados através de matrizes de objectos

6.2 Como criar uma matriz de objectos

Para criar uma matriz de objectos é preciso recordar que “Todos os elementos de uma matriz possuem o mesmo tipo de dados”. Tratando-se de uma matriz de objectos, significa que cada um dos elementos da matriz de objecto é um objecto como tal.

Se considerarmos a classe **Cliente** da lição anterior, podemos criar uma matriz de 5 clientes fazendo:

```
Cliente C[ ] = new Cliente [5];
```

Como resultado, 5 espaços de memória do tipo cliente serão criados um a seguir ao outro como mostra a figura a seguir:

0	1	2	3	4
C [0] ID: Nome: Pagamento:	C [1] ID: Nome: Pagamento:	C [2] ID: Nome: Pagamento:	C [3] ID: Nome: Pagamento:	C [4] ID: Nome: Pagamento:

E para inicializar o cliente 1 por exemplo faríamos:

```
C [1].ID = 5555; C[1].Nome="ANA"; C[1].Pagamento=1600;
```

Contudo, como foi discutido na lição 7, a melhor forma de inicializar uma matriz é através de um ciclo de repetição. Deste modo, seria necessário fazer o mesmo com a matriz de objecto, onde seria necessário o loop para inicializar e outro para visualizar os elementos.

```
Scanner T = new Scanner(System.in);
Cliente C[ ] = new Cliente [5];
for(int i=0; i<5; i++){
    System.out.println("Preenchendo dados do cliente"+i);
    System.out.println("Preenchendo dados do cliente"+i);
    C[ i ].ID = T.nextInt();
    C [ i ].Nome = T.next();
    C [ i ].Pagamento = T.nextFloat();
}
for (int j=0; j<5; j++)
{
    System.out.println("Dados do cliente"+i);
    System.out.println("ID: "+ C [ j ].ID);
    System.out.println("Nome: "+ C [ j ].Nome);
    System.out.println("Pag.: "+ C [ j ].Pagamento);
}
```

6.3 Caso de Estudo



Pretende-se desenvolver um programa que permite armazenar dados de 5 Funcionários, onde cada um possui ID, Nome e Salário. O sistema deve permitir inserir dados e visualizar a soma dos salários dos 5.

Solução

```
import java.util.Scanner;
classe Funcionario {
    int ID; String Nome; float Sal;
    public static void main(String args[]){
        Funcionario F[ ] = new Funcionario [5];
        Scanner T = new Scanner(System.in);
        float S = 0;
        for(int i=0; i<5; i++){
            System.out.println("Preenchendo dados do Funcionario"+i);
            F[ i ].ID = T.nextInt();
            F [ i ].Nome = T.next();
            F [ i ].Sal = T.nextFloat();
            S += F [ i ] . Sal; //Acumulando o salario de cada um dos funcionarios
        }
        System.out.println("Soma dos salários"+S);
    }
}
```

Nesta proposta, foi criado um ciclo de repetição que permite inserir os dados de cada um dos objectos da matriz, onde a cada inserção é acumulado o valor em S.

No fim o valor de S visualizado.

LIÇÃO 7: PACOTES

7.1. Introdução

Toda a linguagem de programação possui suas próprias bibliotecas onde cada uma é adicionada ao programa de acordo com as necessidades. A linguagem java organiza suas bibliotecas em pacotes, onde cada pacote possui uma série de classes. sendo assim, na presente lição iremos abordar os pacotes existentes na linguagem Java e depois iremos criar nossos próprios pacotes.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Explicar os principais pacotes existentes na linguagem Java
- Pesquisar as classes e métodos existentes em Pacotes Java
- Criar pacotes e aceder aos métodos de suas classes

7.2 Conceito de Pacotes

Os pacotes são um conjunto de classes contidas dentro de uma pasta, e interligadas a tal pasta através da expressão “package”.

Assim, classes com funcionalidades comuns podem ser agregadas numa só unidade, permitindo organizar os programas de forma mais simples.

A linguagem Java possui uma variedade de pacotes que podem ser importados dependendo do que se pretende.

Veja a figura abaixo:

Introdução à Programação Orientada a Objectos

Java Packages

LANG	AWT	UTIL	SQL	NET
• String • Integer • Math	• <u>TextField</u> • Button • Frame	• Timer • Scanner • Random	• Connection • Statement • <u>ResultSet</u>	• <u>InetAddress</u> • Socket • URL

Por exemplo o pacote **Lang** possui as classes como String, Integer, Math, o pacote **sql** possui classes ligadas à gestão de uma base de dados como Socket, InetAddress e assim por diante.

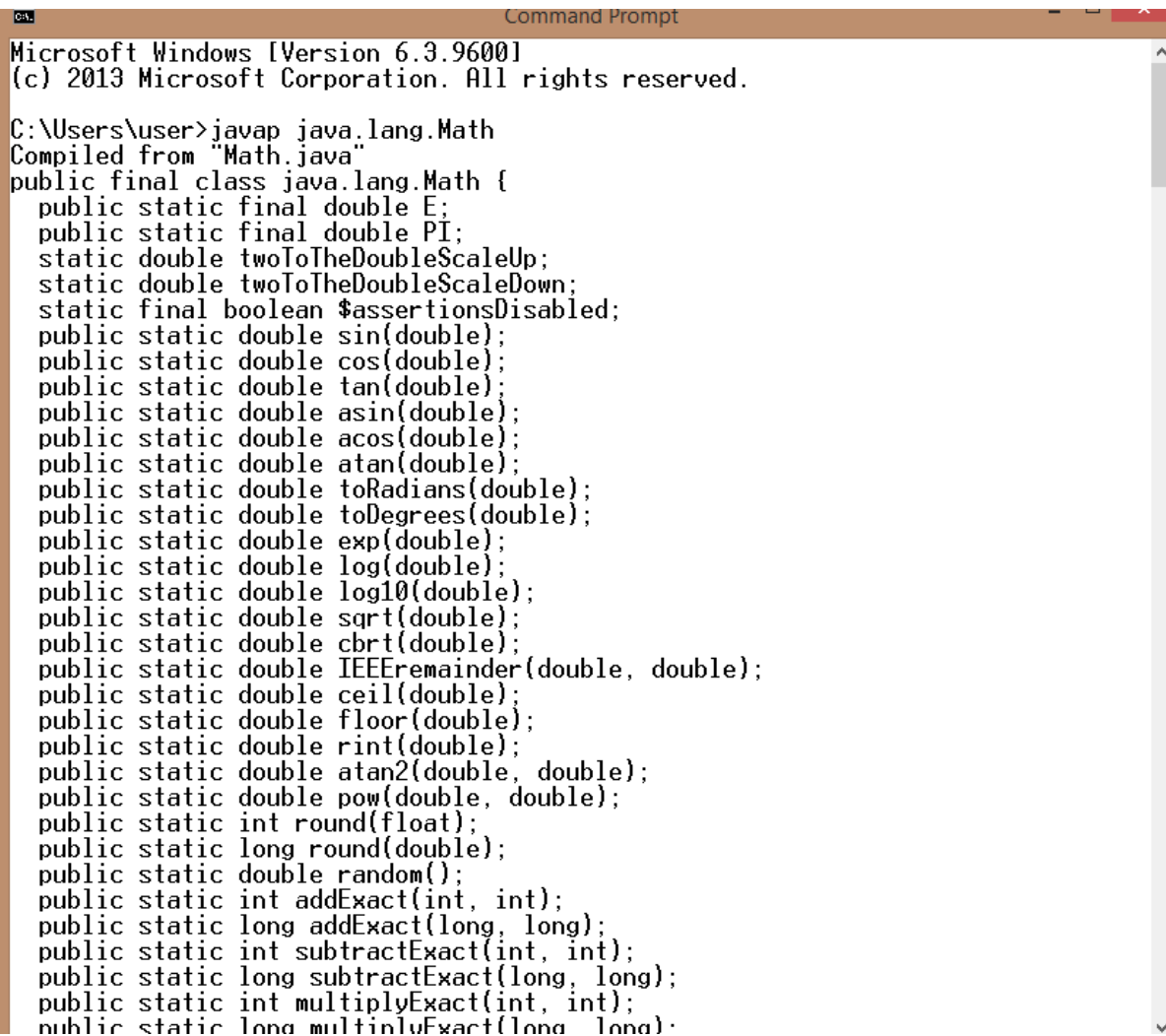
Para visualizar os métodos duma classe em contida em um pacote usamos a instrução **javap** seguido do nome do pacote e do nome da classe (no command prompt do Windows ou no terminal do Linux).



Por exemplo, se pretendemos visualizar os métodos da classe Math usamos a seguinte instrução:

javap java.awt.String

E como resultado teremos o seguinte:



```

Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Users\user>javap java.lang.Math
Compiled from "Math.java"
public final class java.lang.Math {
    public static final double E;
    public static final double PI;
    static double twoToTheDoubleScaleUp;
    static double twoToTheDoubleScaleDown;
    static final boolean $assertionsDisabled;
    public static double sin(double);
    public static double cos(double);
    public static double tan(double);
    public static double asin(double);
    public static double acos(double);
    public static double atan(double);
    public static double toRadians(double);
    public static double toDegrees(double);
    public static double exp(double);
    public static double log(double);
    public static double log10(double);
    public static double sqrt(double);
    public static double cbrt(double);
    public static double IEEEremainder(double, double);
    public static double ceil(double);
    public static double floor(double);
    public static double rint(double);
    public static double atan2(double, double);
    public static double pow(double, double);
    public static int round(float);
    public static long round(double);
    public static double random();
    public static int addExact(int, int);
    public static long addExact(long, long);
    public static int subtractExact(int, int);
    public static long subtractExact(long, long);
    public static int multiplyExact(int, int);
    public static long multiplyExact(long, long);
}

```

Deste modo, é possível perceber como está estruturado por exemplo o método **public static double sin(double)**, onde percebe-se que este é estático (pode ser acedido através da classe), é não void e recebe uma variável **double** como parâmetro.

Exemplo 2: Pretende-se utilizar a classe alguns métodos da classe **Scanner** de modo a receber os dados pelo teclado.

O primeiro passo consiste a verificar os métodos da classe usando o comando:

javap java.lang.String

Será usado o método **nextInt()**;

```
public java.lang.String findInLine(java.util.regex.Pattern);
public java.lang.String findWithinHorizon(java.lang.String, int);
public java.lang.String findWithinHorizon(java.util.regex.Pattern, int);
public java.util.Scanner skip(java.util.regex.Pattern);
public java.util.Scanner skip(java.lang.String);
public boolean hasNextBoolean();
public boolean nextBoolean();
public boolean hasNextByte();
public boolean hasNextByte(int);
public byte nextByte();
public byte nextByte(int);
public boolean hasNextShort();
public boolean hasNextShort(int);
public short nextShort();
public short nextShort(int);
public boolean hasNextInt();
public boolean hasNextInt(int);
public int nextInt();
public int nextInt(int);
public boolean hasNextLong();
public boolean hasNextLong(int);
public long nextLong();
public long nextLong(int); ...
```

```
//Importando a classe Scanner do pacote util
import java.util.Scanner
class A{
    public static void mani(String args[])
    {
        Scanner T = new Scanner(System.in);
        System.out.print("Digite seu nome");
        String n=T.next();
        System.out.print("Ola" + n);
    }
}
```

Neste programa foi possível importar a classe Scanner do pacote util. Poderíamos apenas importar todas as classes do pacote util usando: **import java.util.***

7.3 Como criar um pacote

Para criar um pacote pessoal em java precisamos seguir os seguintes passos:

- 1 - Cria uma pasta com o nome do pacote, por exemplo pac1
- 2 - Dentro do directório crie uma classe qualquer indicando o nome do pacote:

```
package pac1;

public class A{

    public void show(){ System.out.print("Ola"); }

}
```

3 - Fora do directório crie uma classe qualquer que importa o pacote pac1 e faz o uso da classe A que está dentro do pacote:

```
import java.pac1.*;

class P
{
    public static void main(String z[])
    {
        A obj = new A();
        obj.show();
    }
}
```

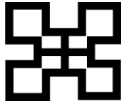
Sendo assim, é possível criar o objecto de A em P ou mesmo fazer a herança de A em P.



JAVA™ PLATFORM, STANDARD EDITION 7 API SPECIFICATION

<https://docs.oracle.com/javase/7/docs/api/overview-summary.html>

Website contendo uma lista de pacotes do Java, e as respectivas classes.



Actividade de auto aprendizagem:

1. Assinale se as seguintes afirmações são verdadeiras(V) ou falsas (F)
- a) Uma classe em Java pode conter vários métodos **main**.
 - b) Podemos ter dois métodos com o mesmo nome dentro duma classe.
 - c) O tipo de retorno (void ou não void) é condição para haver sobrecarga.
 - d) Não podemos ter mais de um construtor numa classe.
 - e) Existem métodos que podem ser acedidos sem a existência de qualquer objecto da classe.
 - f) Um método declarado como estático, só pode ser acedido por outro método estático.
 - g) O main() pode aceder a qualquer método estático.
 - h) Numa matriz de objectos é obrigatório inicializar cada um dos elementos desta usando o comando **new**.

Respostas

- a) Falso: pois uma única classe só deve conter um único main.
- b) Verdadeiro: usando a sobrecarga é possível
- c) Falso: A sobrecarga manifesta-se pela diferença nos parâmetros e não no tipo de retorno.
- d) Falso: Podemos usar a sobrecarga de construtores de modo a ter mais de um construtor na classe.
- e) Verdade: Os métodos estáticos podem ser acedidos via objecto ou através do nome da classe.
- f) Verdade: Métodos estáticos só podem aceder outros métodos estáticos.
- g) Verdade: porque o main é também estático.
- h) Verdade: Pois cada um dos elementos é um objecto que precisa ser inicializado.

[FIM DA UNIDADE III]

UNIDADE TEMÁTICA IV Herança

Introdução geral

Na lição 2 introduzido o conceito de herança como um dos princípios OOP, onde discutimos que esta consiste na capacidade de uma classe herdar os membros da outra. Nesta unidade, pretendemos aprofundar este princípio, onde serão discutidos os principais tipos de herança, o conceito da sobreposição e classes abstractas.

Assim, na lição 1 iremos discutir o conceito de herança e os seus tipos, na lição 2 será abordado o princípio da sobreposição, na lição 3 iremos discutir o conceito de interfaces e herança múltipla, na lição 4 as classes abstractas e na lição 5 iremos abordar as visibilidades (private, protected e friendly).

LIÇÃO 1: CONCEITO DE HERANÇA E SEUS TIPOS

1.1. Introdução

A herança consiste na habilidade que uma classe possui de herdar as características de outra classe, onde a classe que herdada é chamada de **super classe/classe base/classe mãe** e a classe que herda a outra é chamada de **sub classe/classe derivada/classe filho**.

Deste modo, a herança permite assim a reutilização do código de uma classe, isto é, chamar um método que se encontra em outra classe (várias vezes).

Assim, na presente lição iremos simular este princípio com base em um caso de estudo a seguir, discutir os principais tipos de herança.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

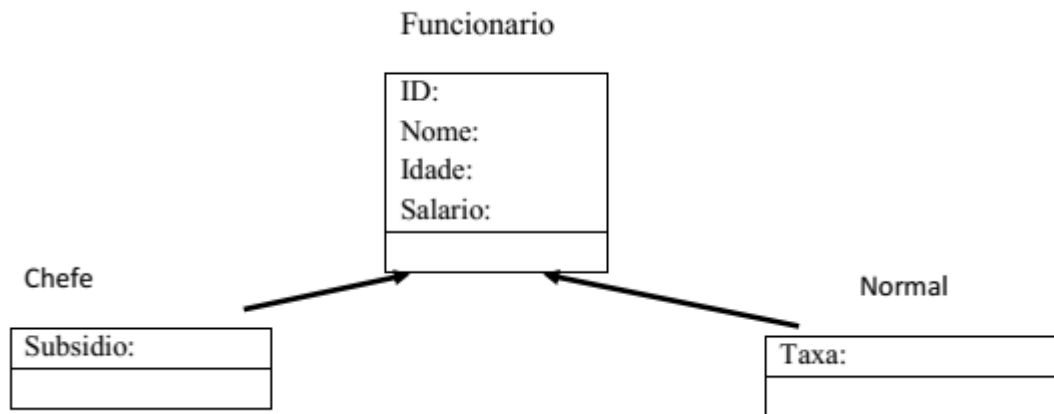
- Simular o princípio de herança de modo a resolver um problema
- Compreender os principais tipos de herança

1.2 Conceito de herança

De modo a efectuar a herança em java usamos a palavra reservada “extends”.



Considere o caso em que pretendemos desenvolver uma aplicação que permite inserir e visualizar dados de uma empresa organizada de forma hierárquica, tal como mostra a figura abaixo:



É possível aqui perceber que Todos os funcionários possuem ID, Nome, Idade e Salário, mas o chefe possui subsídio e o Normal possui a Taxa.

A seguir a proposta de solução:

```
class Funcionario{
    public int ID;
    public String Nome;
    public int Idade;
    public double Salario;
}
class FChefe extends Funcionario{
    public int subsidio;
}
class FNormal extends Funcionario{
    public int taxa;
}
class Programa{
    public static void main(String args[])
    {
        FChefe F1=new FChefe();
        F1.Nome="Jafete";
        F1.subsidio=1000;
        F1.Idade=35;
        FNormal F2=new FNormal();
        F2.Nome="Julia";
        F2.taxa=1250;
    }
}
```

O Objecto F1 do tipo FChefe só pode aceder aos dados/métodos de Funcionário e seus próprios dados que é o subsídio. O mesmo acontece com o F2 que só pode aceder aos dados do Funcionário mais a taxa.

Contudo, se criamos um objecto do tipo funcionário, este não pode aceder a nenhum dado da sua subcasse pelo a instrução:

```
Funcionario F3 = new Funcionario();
```

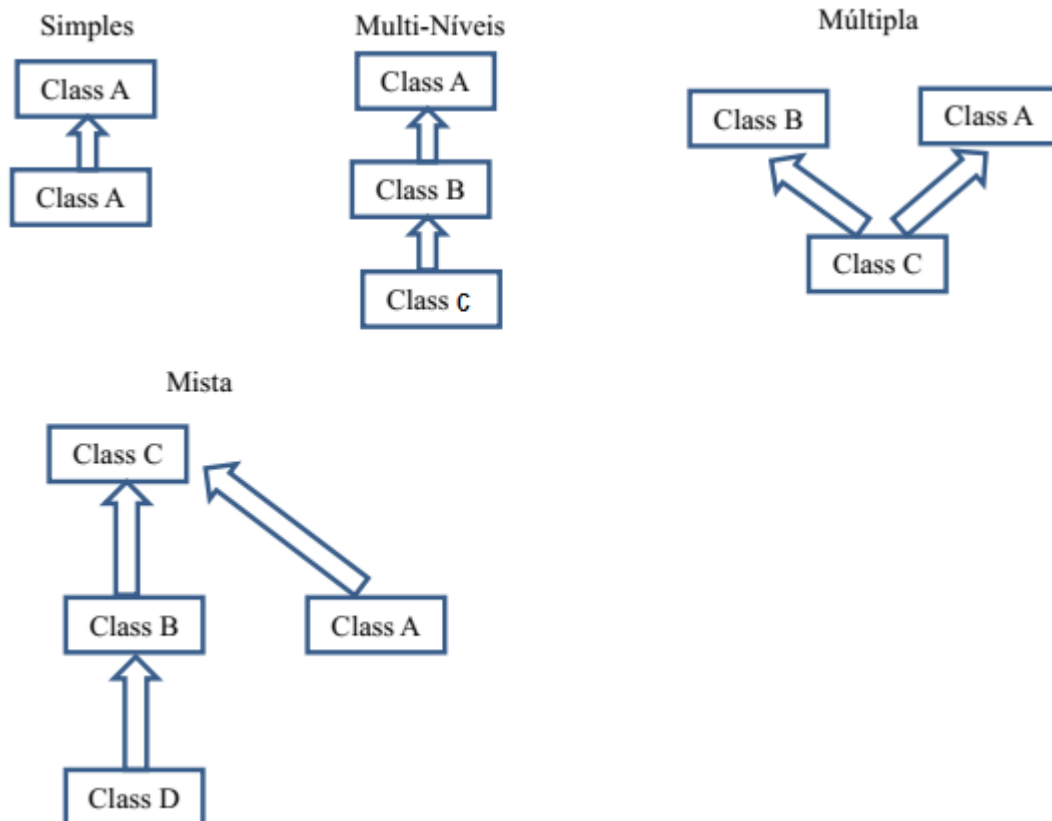
```
F3.taxa=10; // ERRO!
```



“O objecto do tipo da classe base/super classe não pode aceder a dados/métodos das suas sub-classes”.

1.3 Tipos de Herança

A combinação entre classes através da herança é importante pois permitem que os sistemas desenvolvidos espelhem a realidade das organizações que muitas das vezes estão organizadas em sectores e hierarquias. Deste modo, a linguagem Java permite os seguintes tipos:



Introdução à *Programação Orientada a Objectos*

A herança simples consiste em apenas uma classe herdando a outra. Na herança multi-nível temos uma hierarquia de três níveis, onde temos a classe C herdando a classe B e a classe B herdando a classe A (uma relação de pai, filho/a e neto/a).



O Java não suporta herança múltipla, pois uma classe não pode fazer o extends de duas ou mais classes, mas sim apenas uma. Para tal, o conceito de interfaces é usado e será discutido na lição seguinte (Interfaces - Herança múltipla).

LIÇÃO 2: SOBREPOSIÇÃO

2.1. Introdução

Na herança simples e herança multinível podem existir casos em que existam dois métodos totalmente iguais (mesmo nome e tipos de argumentos), estando um método na classe base e o outro na classe derivada. A este princípio chamamos de “sobreposição”. A sobreposição é o contrário da sobrecarga (discutida na unidade III) pois pressupõe-se que os dois métodos sejam realmente iguais, contudo localizados em diferentes classes.

Assim, na presente lição iremos discutir e exemplificar a sobreposição, bem como implementar técnicas que permitam distinguir os dois métodos sobrepostos.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Demonstrar o princípio da sobreposição em OOP
- Resolver o problema da sobreposição usando a palavra reservada.

2.2 Sobreposição

Nos casos em que existem dois métodos totalmente iguais, estando um na **super** classe e outro na **sub** classe e criarmos um objecto do tipo da classe derivada qual dos métodos será invocado visto que os dois são iguais?

No programa que se segue iremos simular a sobreposição, criando dois métodos **void Show()**, onde um estará na classe Mae e outro na classe filho.

O que irá acontecer se criarmos um objecto do tipo filho e chamarmos o método **show()**? tendo em conta que o objecto do tipo filho possui os dois métodos.

```

class Mae{
    public void show() {System.out.print("Sou super classe"); }
}
class Filho extends Mae {
    public void show() {System.out.print("Sou classe filho"); }
    public static void main(String args[])
    {
        Filho F = new Filho();
        F.show(); //Qual dos Shows será invocado?
    }
}

```

Ao executar o programa o resultado será:

```

C:\Users\csengo\Desktop\java\apps\heranca>java Filho
Sou classe filho
C:\Users\csengo\Desktop\java\apps\heranca>_

```

Como esta patente, o método show da classe Filho sobrepõe o show da classe Mae. A este conceito nós chamamos de “sobreposição ou override”. É muito importante discuti-lo pois em java existem classes chamadas de abstractas que só possuem métodos que devem ser obrigatoriamente ser sobrepostos. Por exemplo, para adicionar o evento clique **ActionEvent** num botão será necessário fazer o override do método **public void actionPerformed**. Mais detalhes serão discutidos na unidade 6 – Introdução à programação GUI.

2.3 Como resolver o problema da sobreposição

Continuando o programa anterior, a solução para chamar o método da classe base (o que foi sobreposto) usamos a seguinte técnica: Criamos um método com nome diferente e que chama o método da superclasse através da palavra reservada **super**.

Vejamos a solução:

```

class Mae{
    public void show() {System.out.println("Sou super classe"); }
}

```

```
class Filho extends Mae {  
    public void show() {System.out.println("Sou classe filho"); }  
    public void mostra(){ super.show(); }  
  
    public static void main(String args[])  
    {  
        Filho F = new Filho();  
        F.show(); //chama o Show() da classe filho  
        F.mostra(); //chama o Show() da classe Mae  
    }  
}
```

Neste caso foi criado um outro método **mostra()** que chama o método **show()** da classe Mae usando a palavra reservada **super**, que é usada para invocar métodos e dados da classe base.

Assim como resultado teremos o seguinte:

```
C:\Users\csengo\Desktop\java\apps\heranca>java Filho  
Sou classe filho  
Sou super classe
```

Assim, foi possível resolver a sobreposição, chamando os dois métodos.

LIÇÃO 3: INTERFACES E HERANÇA MÚLTIPLA

3.1. Introdução

Como foi dito, Java não suporta herança múltipla e para tal faz o uso do conceito de interfaces. Interface é uma classe como qualquer outra, contudo os seus dados são constantes (não mudam o seu valor) e os seus métodos não tem corpo, sendo necessário fazer a herança e a sobreposição destes para poder usa-los.

Na presente lição iremos discutir como programar as interfaces e como implementar a herança múltipla usando interfaces.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Perceber o conceito de interface em herança e sobreposição
- Utilizar o conceito de interface para implementar a herança múltipla

3.2 Interfaces e a sua sobreposição

Antes de criar uma interface, é necessário perceber as regras básicas:

- Os dados em uma interface são constantes.
- Os métodos não tem corpo, apenas a definição.
- Podem fazer o **extends** de várias interfaces
- Não podem ser instanciadas, isto é, não podem ter objectos.

Criemos a nossa primeira interface, que na verdade é uma classe simples que contem alguns métodos:

```
interface Mae{  
    public void Mostrar();  
}
```

O passo seguinte consiste em criar uma outra classe que irá fazer a herança da interface, usando a palavra reservada **implements** (ao invés do “extends”). Logo a seguir cabe a classe derivada fazer a sobreposição de todos os métodos da interface por ele implementada.

```
class Prog implements Mae{
    public void Mostrar(){
        System.out.println("Olaaaa!");
    }
    public static void main(String args[]){
        Prog obj = new Prog();
        obj.show();
    }
}
```

Sendo assim, a classe Prog fez a sobreposição da interface Mae, cabendo a esta fazer a sobreposição de todos os métodos da interface por ela herdada.

3.3 Herança múltipla usando interfaces

Como foi vista na lição anterior, uma classe não pode herdar (extends) mais de uma classe, contudo, uma classe pode implementar (implements) mais de uma interface, tornando possível a herança múltipla.

No exemplo a seguir, iremos demonstrar como simular a herança múltipla, onde teremos uma classe Filho a herdar duas classes, a classe Mae e classe Pai.

Ora vejamos:

```
interface Mae{
    public int ID=100;
    public void show();
}
interface Pai{
    public int ID=111;
    public void imprime();
}
class Filho implements Mae,Pai {
    public void show() {System.out.println("Sou classe Mae"); }
    public void imprime(){ System.out.println("Sou classe Pai"); }
    public static void main(String args[])
    {
        Filho F = new Filho();
    }
}
```

```
    F.show();  
    F.imprime();  
}  
}
```

A classe filho faz a herança das classes Mae e Pai e faz o override dos métodos show e imprime, permitindo assim que uma única classe faça a herança de mais de uma classe (herança múltipla).



"É da responsabilidade da classe derivada fazer a sobreposição de todos os métodos das interfaces nas quais ela fez a herança (implements)."

LIÇÃO 4: CLASSE ABSTRACTA

4.1. Introdução

Na lição anterior vimos que uma interface é uma classe que deve ter subclasses para poder existir e os métodos são apenas declarados (sem o corpo). Contudo, por vezes pretendemos que uma classe normal se comporte como uma interface. A esse tipo de classe chamamos de “classe abstracta”, usando a palavra reservada “abstract”.

Assim, nesta lição iremos discutir o conceito de classes “*classes abstractas*”, como cria-las e a sua importância.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Implementar o conceito de classes abstractas na resolução de problemas.
- Compreender a importância das classes abstractas na organização de um programas.

4.2 O conceito classe abstracta

Uma classe abstracta possui características próprias (semelhantes às interfaces), que iremos mencionar a seguir:

- Não podemos instanciar (criar objectos)
- Os detalhes dos métodos abstractos serão conhecidos na subclasse (a quando da sua sobreposição).
- Não podemos declarar construtores ou membros estáticos em classes abstractas.
- Se uma classe normal possui pelo menos 1 método abstracto, então tal classe deve ser obrigatoriamente abstracta.

Criando uma classe abstracta:



Consideremos uma aplicação de cadastro de Viaturas, onde cada viatura possui Matrícula, Marca, KM (a quilometragem), o método Acelerar() que incrementa a KM.

Podemos assim criar uma classe abstracta que contém a estrutura da aplicação, e depois criar a sua subclasse que poderá conter os detalhes de implementação da classe, como mostra a proposta a seguir:

```
abstract class Viatura{
    public String Matricula, Marca;
    public float KM;
    public abstract void iniciar(String M, String Mc, float K);
    public abstract void Acelerar();
}

class Detalhes extends Viatura{
    public void Acelerar()
    {
        KM = KM + 10;
    }
    public void iniciar(String Mat, String Marc, float km)
    {
        Matricula = Mat;
        Marca = Marc;
        KM = km;
    }

    public static void main(String f[])
    {
        Detalhes obj = new Detalhes();
        obj.iniciar("MMA-11-90","Toyota",0F);
        obj.Acelerar();

        System.out.println("Marca:"+obj.Marca+"Matricula:"+obj.Matricula+"Quilometragem:"+obj.KM);
        obj.Acelerar();

        System.out.println("Marca:"+obj.Marca+"Matricula:"+obj.Matricula+"Quilometragem:"+obj.KM);
    }
}
```

Na classe abstracta Viatura foram declaradas os dados: Matrícula, Marca e KM, e os métodos **iniciar()** e **acelerar()** que como pode ver, não possui corpo, contendo apenas a definição, pois estes métodos são abstractos. Logo a seguir, a classe

Introdução à Programação Orientada a Objectos

Detalhe faz a herança de Viatura, devendo esta sobrepor todos os métodos presentes na classe abstracta.

No **main()**, é criado o objecto da subclasse e através deste são chamados os métodos da super classe, o **iniciar()** e **acelerar()**.

Assim, A classe viatura contém apenas a estrutura básica da classe (declaração de dados e métodos) e a classe Detalhes contém a codificação ou os detalhes da implementação dos métodos.

Este processo facilita a organização do programa, pois as duas partes ficam separadas, facilitando a detenção de erros, bem como a leitura do programa.

LIÇÃO 5: VISIBILIDADE (PRIVATE, PUBLIC, PROTECTED)

5.1. Introdução

Até então vínhamos desenvolvendo classes onde todos os membros tinham a visibilidade pública (acessíveis para todos). Contudo, de modo a garantir a abstracção e encapsulamento de dados (discutidos na unidade 1) é necessário proteger alguns membros, de modo que só possam ser acedidos dentro da classe ou apenas nas classes derivadas, ou ainda, dentro ou fora do pacote. Vale notar que alguns métodos por exemplo só dizem respeito ao funcionamento interno da classe e não precisam ser declarados como “public”.

Assim, nesta lição iremos discutir as diferentes visibilidades dos membros (public, protected, private, friend) de forma a proteger a classe de acessos externos.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Compreender os diferentes tipos de visibilidade presentes nas linguagens OOP
- Implementar as diferentes visibilidades na proteção da classe
- Perceber a técnica de acesso a membros que estejam protegidos.

5.2 Visibilidade em Java

Quando declaramos um membro como “public” significa que este é visível em qualquer parte do programa, sendo que, qualquer objecto dentro ou fora do pacote pode acede-lo bastando criar objecto da classe. Entretanto, há situações em que pretendemos limitar o acesso, de modo que este seja acessível apenas dentro da classe ou mesmo apenas na subclasse. Assim, java possui outras visibilidades como o **protected**, **friendly** e **private**.

A seguir iremos abordar cada uma delas:

Friendly

Na visibilidade pública os objectos são acessíveis em outras classes (dentro ou fora do pacote), contudo, quando nenhuma visibilidade é mencionada o membro toma a visibilidade friendly/default. Estes membros, são acessíveis dentro da classe, na subclasse e em outras classes, dentro do pacote.



Por exemplo:

```
class A {
    int VA;
}
```

A variável VA é friendly, e só pode ser acedida por subclasse ou outras classes (criando objecto) que estejam dentro do mesmo pacote em que a classe A está.

Protected

Esta visibilidade situa-se entre a “friendly” e a “public”. Quando um membro é declarado como “protected” passa a ser visível dentro da classe e subclasse no mesmo pacote, mas também passa a ser acessível dentro duma sub classe que está em outro pacote.



Por exemplo:

Consideremos as seguintes classe, cada uma no seu pacote:

Pacote1	Pacote2
<pre>class A{ protected void show(){ ...} }</pre>	<pre>class B extends A{ } class C { }</pre>

Se a classe B é subclasse de A e está em outro pacote, pode também aceder o método show(). Contudo, a classe C, mesmo que crie objecto de A, não poderá aceder ao método show().

Private

A visibilidade privada constitui o nível mais elevado de protecção em OOP, uma vez que um membro declarado como privado é acessível somente dentro da sua classe, isto é, as subclasses ou outras classe, mesmo que estejam no mesmo pacote, não podem aceder a dados privados.



Por exemplo: Considere a classe seguinte:

```
class A {  
    private String Senha;  
    public void inserir() { Senha = 0; }  
}  
  
class B extends A{ ... }
```

Na classe acima, o dado **Senha** é privado e só pode ser acedido por outros membros que estejam dentro da classe A. Por outro lado, a classe B não pode aceder à Senha, apesar de ter feito herança de A, pois, um membro tipo privado não é se quer herdado.

Portanto, sempre que pretendemos esconder uma classe ou membro, declaramos como “private”, passando a ter vida dentro da classe apenas.

5.3 Como aceder a membros não acessíveis

Como pudemos ver, as visibilidades permitem limitar o acesso aos membros da classe tanto dentro ou fora da classe. No entanto, mesmo que um membro não seja acessível, existe uma forma de torna-lo acessível. Para tal, criamos um método público na zona acessível que poderá aceder tal membro, e usamos este método na zona proibida, pois este é público.



Por exemplo: Considere a classe seguinte:

```
class A {    private int VA; }
class B extends A{
}
```

Como tínhamos discutido anteriormente, a classe B não pode aceder ao dado **VA**.

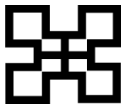
Para tornar isto possível, criamos dois métodos públicos, onde o primeiro será usado para inicializar o valor de VA e o segundo será usado para visualizar o valor de VA. Estes métodos devem ser criados dentro da classe A, que é a única parte onde o VA é acessível.

```
class A {    private int VA;
    public void inserirVA(int x) { VA = x; } // método para inicializar VA
    public int mostraVA() { return VA; } // método para visualizar VA
}
class B extends A{
}
```

Assim a classe B pode usar os métodos **inserirVA()** e **mostrarVA()** que são públicos para manipular VA de forma indirecta como mostra a solução final abaixo:

```
class A {    private int VA;
    public void inserirVA(int x) { VA = x; } // método para inicializar VA
    public int mostraVA() { return VA; } // método para visualizar VA
}
class B extends A{
    public static void main (String args[])
    {
        B obj = new B();
        obj.inserirVA(50);
        System.out.println("VA="+obj.mostrarVA());
    }
}
```

Assim, mesmo que um membro seja privado, podemos torna-lo acessível, contudo, ele continua protegido, pois não pode ser acedido directamente.



Actividade de auto aprendizagem

1 – Explique os seguintes conceitos:

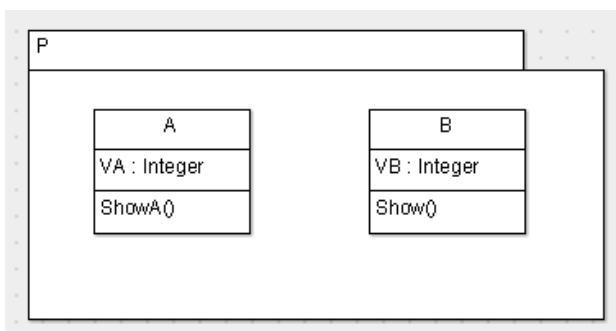
- Herança
- Static
- pacotes

2 – Considere a seguinte classe:

Banco

<u>ID:</u>
Nome:
capital
<u>VerDados()</u>
<u>InserirDados()</u>

- a) Complete os métodos da classe e crie três Bancos(objectos) dela
- b) Imprima a soma dos capitais dos três Bancos.
- c) Crie um construtor que inicializa os dados através de parâmetros.
- d) Crie uma classe Principal e faça a herança da classe Banco, fazendo por seguinte o uso de todos os seus métodos no main.
- 3 - Crie um pacote P e nele crie e compile duas classes A e B tal como ilustra a figura abaixo:



A seguir Crie uma outra classe Principal (fora do pacote P) que deve fazer a importação de P e fazer o uso das classes A e B. A classe A deve ser acedida através da herança e a classe B via objecto.

Respostas:

1 - Herança: Capacidade de uma classe herdar métodos e dados de outra.

Static: Pertence à classe e é partilhado pelos objectos

Instância: Objecto

Pacote: Conjunto de classes agregadas em uma unidade (pasta)

Banco

<u>ID:</u>
Nome:
capital
<u>VerDados()</u>
<u>InserirDados()</u>

2 – a),b) e c)

```
import java.util.Scanner;
class Bancos{
    public String ID;
    public String Banco;
    public float capital;

    public void VerDados(){
        System.out.print(ID+Banco+capital);
    }
    public Bancos(String id, String b, float c){
        ID = id; Banco = b; capital=c;
    }
    public void InserirDados(){
        Scanner T = new Scanner(System.in);
        ID = T.next(); Banco=T.next(); capital=T.nextFloat();
    }
    public static void main(String[] args) {
        Bancos b1 = new Bancos(); b1.VerDados();
        Bancos b2 = new Bancos(); b2.VerDados();
        Bancos b3 = new Bancos(); b3.VerDados();
        float R = b1.capital+b2.capital+b2.capital;
        System.out.println("Soma dos capitais:"+R);
    }
}
```

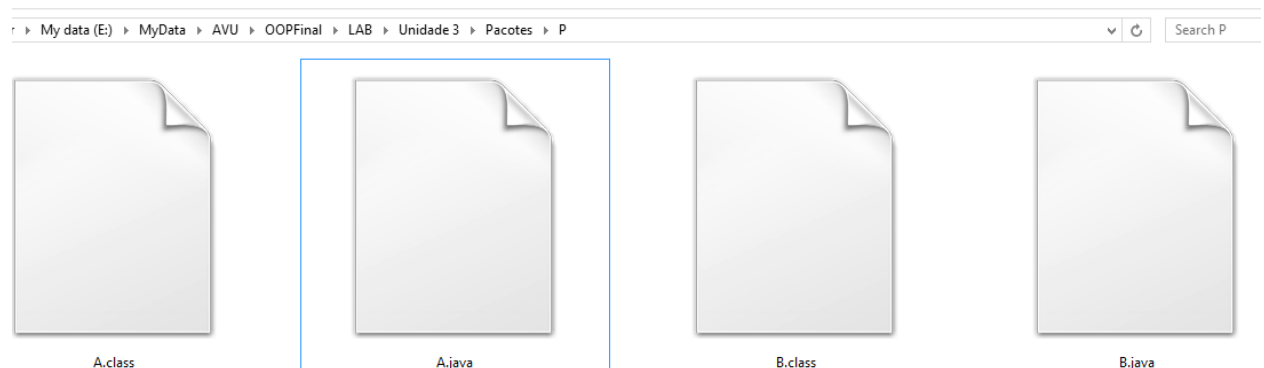
d)

```
class Bancos{
    public String ID;
    public String Banco;
    public float capita;

    public void VerDados(){
        System.out.print(ID+Banco+capital);
    }
    public void InserirDados(){
        Scanner T = new Scanner(System.in);
        ID = T.next(); Banco=T.next(); capital=T.nextFloat();
    }
}
class Principal extends Bancos{
    public static void main(String[] args) {
        Principal b1 = new Principal(); b1.VerDados();
        Principal b2 = new Principal(); b2.VerDados();
        Principal b3 = new Principal(); b3.VerDados();
    }
}
```

3 – Proposta de solução

Passo 1: Crie uma pasta P e nela crie as seguintes classes em separado:



<pre>package P; public class B{ public int VB; public void showB(){</pre>	<pre>package P; public class A{ public int VA; public void showA(){</pre>
--	--

Introdução à Programação Orientada a Objectos

<pre> System.out.println("Eu sou da classe B"); } }</pre>	<pre> System.out.println("Eu sou da classe A"); } }</pre>
--	--

Passo 2: A seguir crie a classe principal fora do directório P e nela faça a importação do pacote P e faça a herança de A. Para a classe B será criado o objecto no main.

```
import P.*;

class Principal extends A{

    public static void main(String args[])

    {    //Chamando A através da herança

        Principal objA = new Principal();

        objA.VA=10; System.out.print("VA="+objA.VA);

        //chamando B via objecto

        B objB = new B();

        objB.VB=15; System.out.print("VB="+objB.VB);

    }

}
```

Output:

```
VA=10VB=15
```

No presente programa foi possível constatar que a classe Principal consegue facilmente aceder aos métodos e dados da classe A via herança e aos métodos e dados de B via objecto.

[FIM DA UNIDADE IV

UNIDADE TEMÁTICA V Tratamento de erros (Excepções)

Introdução geral

Os programas escritos na linguagem de programação devem ser compilados antes da sua execução, de modo a eliminar os possíveis erros de sintaxe e semântica da linguagem. Assim, mesmo após uma compilação bem-sucedida podem existir ainda que só podem ser detectados durante a execução do programa tal como a divisão por zero. Sendo assim, a presente unidade visa capacitar os alunos sobre a gestão dos erros de execução, o que em programação chamamos de tratamento de erros e excepções.

Assim, na lição 1 iremos discutir o conceito de excepções e erros, na lição 2 será discutido a técnica de criação de excepções.

LIÇÃO 1: TRATAMENTO DE ERROS E EXCEPÇÕES GENÉRICAS

1.1. Introdução

Durante o processo de programação podemos nos deparar com 2 tipos de erros: Erros de compilação e erros de execução. Os erros de compilação são os que violam a sintaxe da linguagem de programação e são facilmente detectados durante a compilação. Os erros de execução são aqueles que produzem resultados inesperados durante a execução do programa (após uma compilação bem sucedida), por exemplo: Quando indicamos um índice de um array que esteja fora dos limites deste. Tais erros são chamados de exceções.

Assim, na presente lição iremos simular exceções da linguagem Java e fazer o seu devido tratamento.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Entender as exceções em OOP de modo a evitar resultados inesperados
- Implementar o bloco try...catch no tratamento de exceções.

1.2 Exceções

Uma exceção constitui uma condição que é causada por um erro de execução, onde quando tal erro ocorre, o Java cria um objecto tipo exceção e dispara (informa que um erro ocorreu). Se tal exceção não for devidamente tratada o programa pode ser interrompido, contudo se esta for tratada, podemos garantir a continuidade do programa.

Na linguagem Java, existem exceções pré-programadas e que podem ser tratadas por nós. Veja lista a seguir:

Excepção	Causa
ArithmeticException	Causada por erros matemáticos tal como divisão por zero

ArrayIndexOutOfBoundsException	índice errado no array (matriz)
FileNotFoundException	Quando se tenta aceder a um ficheiro que não existe
NullPointerException	Quando se tenta aceder a um objecto vazio (que não tenha sido inicializado)
NumberFormatException	Quando a conversão entre textos e números ocorre de forma incorrecta
Exception	Todos os erros que forem a ocorrer

1.2 Tratamento de Excepções

O tratamento de excepções é efectuado através do bloco **try ...catch** tal como mostra a figura que se segue:

```
try {
  Lista de instruções
}
catch()
{
  Instruções que irão tratar do erro (dizendo o que fazer após a detenção do erro)
}
finally{
  Continuação do programa, havendo ou não tratamento de erro.
}
```

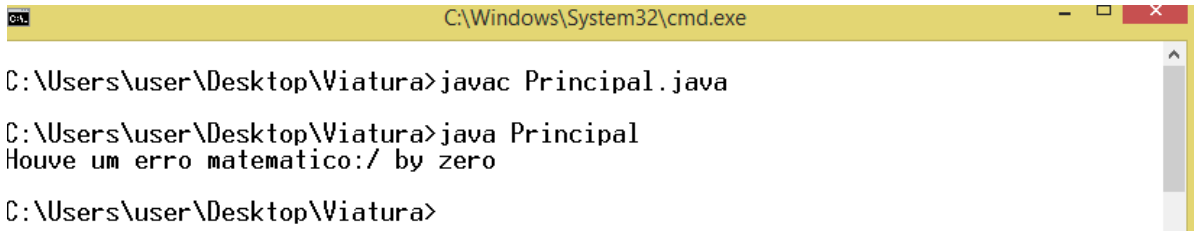
Assim, Suponhamos que pretendemos simular o tratamento do erro da divisão por Zero. Iremos criar um programa que simula a divisão de 50 por 0, e fazer o devido tratamento de erros.

Assim será o programa:

```
class Principal{
    public static void main(String f[])
    {
        try{
            int x = 50;
            float R = x/0; // Erro, divisão por Zero, Excecao sera disparada
            System.out.println("Resultado"+R);
        }
        catch(ArithmeticException e)
        {
            System.out.println("Houve um erro
matematico:"+e.getMessage());
        }
    }
}
```

```
}
}
```

Como resultado teremos:



```
C:\Windows\System32\cmd.exe
C:\Users\user\Desktop\Viatura>javac Principal.java
C:\Users\user\Desktop\Viatura>java Principal
Houve um erro matematico:/ by zero
C:\Users\user\Desktop\Viatura>
```

Ao executar o programa, a instrução **float R = x/0;** causa uma excepção matemática devido à divisão do x por 0. Como resultado, a instrução remanescente **System.out.println("Resultado"+R);** foi ignorada, tendo o controle passado para o catch.

Assim, foi devidamente tratado o erro, com uma mensagem mais simples e direccionada ao erro em causa.



Para um único bloco **try** podemos ter vários tratamentos de erros (**catch**), comportando-se como a instrução **switch** discutida na unidade 2.

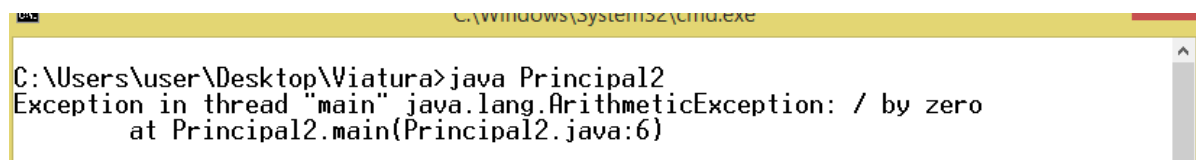
```
try {
  Lista de instruções
}
catch( ArithmeticException e1)
{
  tratamento de excepções matemáticas
}
catch( NumberFormatException e2)
{
  tratamento de excepções de conversão
}
catch( Exception e3)
{
  tratamento de excepções genéricas
}
finally{
  Continuação do programa, havendo ou não tratamento de erro.
}
```

1.2 A instrução “throws”

Existem situações em que pretendemos que as exceções sejam tratadas de forma automática, sem que tenhamos usado o bloco **try...catch**. Nestes casos usamos a instrução **throws** logo a seguir à declaração do método, tal como mostra a instrução a seguir:

```
class Principal2{
    public static void main(String f[]) throws Exception
    {
        int x = 50;
        float R = x/0; // Erro, divisão por Zero, Excepcao sera disparada
e as instruções seguintes ignoradas
        System.out.println("Resultado"+R);
    }
}
```

Assim, como resultado teremos o seguinte:

A screenshot of a Windows command prompt window. The title bar shows the path 'C:\windows\system32\cmd.exe'. The command prompt shows the command 'C:\Users\user\Desktop\Viatura>java Principal2' and the resulting exception: 'Exception in thread "main" java.lang.ArithmeticException: / by zero at Principal2.main(Principal2.java:6)'.

```
C:\windows\system32\cmd.exe
C:\Users\user\Desktop\Viatura>java Principal2
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Principal2.main(Principal2.java:6)
```

LIÇÃO 2: DISPARANDO SUA PRÓPRIA EXCEPÇÃO (THROW)

1.1. Introdução

Na lição anterior vimos que era possível prever e tratar os erros que ocorrem durante a execução do programa, através das exceções pré definidas pela linguagem de programação. Contudo, você como programador, pode criar a sua própria biblioteca de exceções, pois a linguagem Java não pode prever todo o tipo de exceções que podem ocorrer, uma vez que algumas estão ligadas ao contexto, como por exemplo a idade de uma pessoa que não pode ser negativa.

Assim, na presente lição pretendemos demonstrar a você como criar suas próprias exceções.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Criar exceções definidas pelo próprio programador
- Implementar programas usando a instrução throws na gestão de erros próprios

1.2 Exceções definidas pelo programador

Exceções podem ser criadas pelo programador, usando a instrução throws.



Suponhamos que você pretende desenvolver uma aplicação onde a idade de uma pessoa não pode ser inferior a zero (de facto na vida real é assim).

Vale lembrar que o Java não tem como prever este tipo de exceções.

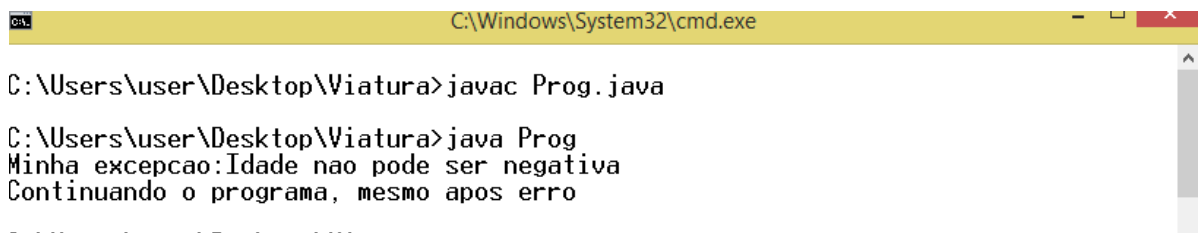
Passo 1: Criamos uma classe responsável por disparar a exceção através do seu construtor:


```
class Erro extends Exception {  
    public Erro(String mensagem){  
        super(mensagem);  
    }  
}
```

Passo2: Criamos a classe principal que irá invocar a classe anterior, fazendo o **throws** seguido da instrução da criação do tal objecto:

```
class Prog{  
    public static void main(String f[]) throws Exception  
    {  
        try{  
            int idade = - 5;  
            if(idade<0)  
                throw new Erro("Idade nao pode ser negativa");  
        }  
        catch(Erro e){  
            System.out.println("Minha excepcao:"+e.getMessage());  
        }  
        finally{  
            System.out.println("Continuando o programa, mesmo apos  
erro");  
        }  
    }  
}
```

Assim o resultado do programa será:

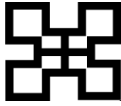


```
C:\Windows\System32\cmd.exe  
  
C:\Users\user\Desktop\Viatura>javac Prog.java  
  
C:\Users\user\Desktop\Viatura>java Prog  
Minha excepcao:Idade nao pode ser negativa  
Continuando o programa, mesmo apos erro  
...
```

Uma vez que a condição **if(idade<0)** é **verdadeira**, a instrução **throw new Erro("Idade nao pode ser negativa");** será executando, transferindo o controle para o catch que irá imprimir **Minha excepção** seguido da mensagem de erro disparada que é **Idade nao pode ser negativa**.

Nota-se o uso do **finally**, que permite executar a instrução **Continuando o programa, mesmo apos erro**, independentemente de ter havido ou não o tratamento de erro.

Assim foi possível criar a nossa própria excepção, usando o **throws**.



Actividade de auto aprendizagem:

Faça a simulação da detenção de erros em arrays (índice fora do intervalo – `ArrayIndexOutOfBoundsException`). Para tal crie um array com dimensão 3 ou mais e tente preencher um elemento com índice 3. Tendo em conta que o índice 3 está fora dos limites do array, o sistema irá compilar com sucesso mas durante a execução irá disparar uma excepção. A tarefa é fazer o tratamento desta excepção usando o bloco `try` e `catch`.

Resposta

Proposta de solução

```
class Principal{
    public static void main(String args[])
    {
        int Arr[] = new int[3];
        try{
            Arr[0]=10;
            Arr[3]=6; //Erro, pois índice 3 está fora dos limites do array
        }
        catch(ArrayIndexOutOfBoundsException e)
        { System.out.print("Indice fora do intervalo do array");}
        catch(Exception e2)
        {System.out.print("Erro qualquer");}
    }
}
```

O programa acima será compilado com sucesso, contudo, durante a sua execução o sistema irá disparar a excepção “`ArrayIndexOutOfBoundsException`” pois o índice 3 está fora dos limites do array (lembrando que se o array tem tamanho 3, então os índices vão de 0 a tamanho-1, neste caso 2).

Output:

Indice fora do intervalo do array

O outro bloco `catch(Exception e2)` será disparado caso ocorra um outro tipo de erro, durante a execução.

[FIM DA UNIDADE V]

UNIDADE TEMÁTICA VI Introdução à Programação GUI

Introdução geral

Nas Unidades anteriores foram discutidos conceitos ligados à programação orientada a objectos com o foco no desenvolvimento de aplicações CLI (Command Line Environment), que são programas executados no ambiente em linha de comando. Contudo, hoje em dia, desde o aparecimento do mouse (rato) as aplicações desenvolvidas passaram desta plataforma para o ambiente gráfico, os vulgos GUI (Graphical User Interface). Este tipo de programas tem como aparência os elementos como: formulários, botões, caixas de texto, listas, entre outros. Deste modo, todos os conceitos previamente discutidos (classes, objectos, herança, etc) constituem pré requisitos para a presente unidade, pois a programação GUI pressupõe a combinação destes elementos todos.

Assim, a presente unidade visa a dotar a você de capacidade técnica para desenvolver aplicações GUI.

Deste modo, na lição 1 serão discutidos os objectos necessários para o desenho de aplicações e na lição 2 serão abordadas as classes necessárias para adicionar eventos (acções) aos objectos desenhados.

LIÇÃO 1: DESENHO DE APLICAÇÕES GUI

1.1. Introdução

A linguagem já java é possui uma variedade de classes próprias que permitem desenvolver uma aplicação GUI (Graphical User Interface) constituída por vários objectos como caixas de textos, botões, listas, etc. O pacote principal é o **awt** que permite desenhar formulários simples, o que mais tarde deu lugar ao pacote **swing** que trouxe mais recursos visuais não disponíveis no seu antecessor.

Assim, na presente lição iremos discutir a forma mais simples de desenhar aplicações para ambientes gráficos.

Objectivo geral da aula/Meta

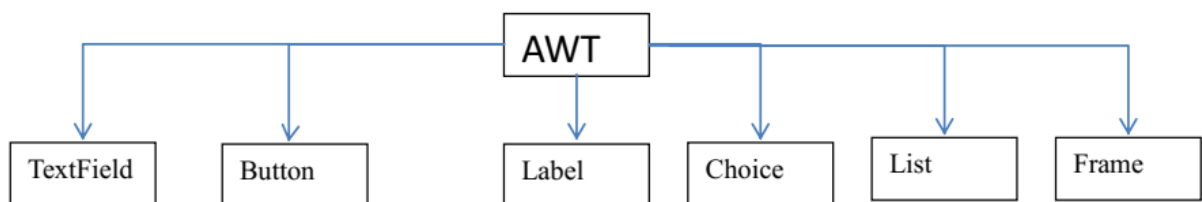


Assim, no fim desta aula o aluno deve ser capaz de:

- Identificar as principais classes usadas para o desenho de aplicações.
- Desenhar aplicações GUI usando as classes principais do pacote awt/swing.

1.2 As principais classes para o desenho GUI

O pacote awt possui as seguintes classes:



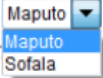
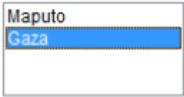
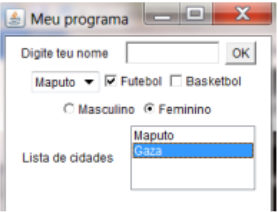
O AWT (Abstract Window Toolkit) é um pacote que contém classes que permitem desenhar imagens e formulários. Este contém também um sub-pacote event que possui uma série de eventos que permitem interagir com o utilizador. Cada uma das classes possui um ou vários construtores e uma variedade de métodos.

A tabela a seguir apresenta alguns detalhes sobre as classes e seus métodos:

Introdução à *Programação Orientada a Objectos*

Classe	Construtor	Métodos	Como criar
TextField	TextField() TextField(String) TextField(String, int) TextField(int)	void setVisible(boolean) void setEnabled(boolean) void setText(String) String getText(String) Char getEchoChar(Char) void setEchoChar(Char) void selectAll() void select(int,int) boolean isEditable(boolean) void setEditable(boolean)	Caixa de texto  Ex: TextField T1; T1=new TextField(10);
Button	Button() Button(String)	void setVisible(boolean) void setEnabled(boolean)	Botão de comando 

Introdução à *Programação Orientada a Objectos*

		void setLabel(String) String getLabel(String)	Ex: Button B; B = new Button("OK");
Label	Label() Label(String) Label(String, int)	void setVisible(boolean) void setEnabled(boolean) void setText(String) String getText(String)	Label de texto Digite teu nome Ex: Label L1; L1 = new Label("Digite o teu nome")
Choice	Choice()	void setVisible(boolean) void setEnabled(boolean) void add(String) String getItem(int) int getSelectedIndex() String getSelectedItem() void select(int)	Caixa de textos (Combobox)  Choice ch; ch = new Choice(); ch.add("Maputo"); ch.add("Sofala");
List	List() List(int) List(int, boolean multi_select)	add(String) delItem(int) removeAll() int countItems(); boolean allowsMultipleSelections(); setVisible(boolean) setEnabled(boolean) getItem(int) getSelectedItem() select(int)	Lista de textos Lista de cidades  Ex: List L; L=new List(); L.add("Maputo"); L.add("Gaza");
Frame	Frame() Frame(String)	void add(Object) void setLayout(Object) void setTitle(String) void resize(int,int)	Janela principal que contém todos os objectos  public class ABC extends Frame { }
Checkbox	Checkbox()	setVisible(boolean)	Checkbox

Introdução à Programação Orientada a Objectos

	Checkbox(String) Checkbox(String, boolean)	setEnabled(boolean) setLabel(String) getLabel(String) getState() setState(boolean)	☒ Futebol ☒ Basketbol Checkbox ck; ck = new Checkbox("Futebol");
Checkbox Group	CheckboxGroup() Checkbox(String, CheckboxGroup, boolean)	setVisible(boolean) setEnabled(boolean) setCheckboxGroup(CheckboxGroup) getCheckboxGroup()	Radio Buttons ☐ Masculino ☒ Feminino genero=new CheckboxGroup(); masc=new Checkbox("Masculino",genero,false); fem=new Checkbox("Feminino",genero,true);



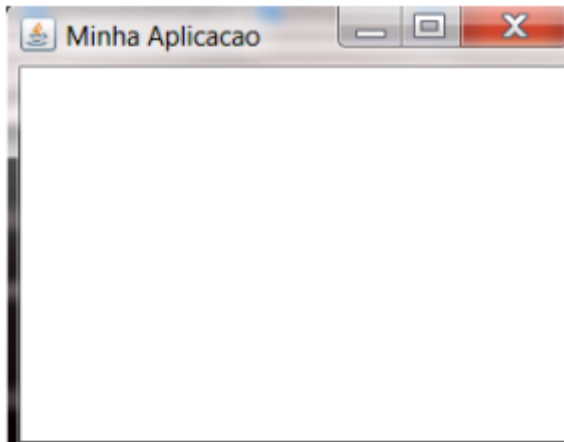
Exemplo 1:

Pretende-se desenhar uma calculadora de dois números introduzidos pelo utilizador.

Passo 1: Desenhamos a janela (Frame)

```
import java.awt.*;
class APP extends Frame
{
    public APP()
    {}
    public static void main(String z[])
    {
        APP form = new APP();
        form.setLayout(new FlowLayout());
        form.resize(400,400); // tamanho
        form.setTitle("Minha Aplicacao"); //titulo da aplicação
        form.show();
    }
}
```

Resultado:



Passo 2: Adicionamos os objectos na classe e inicializados no construtor da classe.

Normalmente declaram-se os objectos na classe e inicializam-se no construtor. Isso vai permitir que as caixas de textos, botões, etc estejam acessíveis para todos os métodos da classe.

```
import java.awt.*;
class APP extends Frame
{
    TextField txtn1,txtn2,txtR;
    Label lbln1,lbln2;
    Button B;
    public APP()
    {
        txtn1 = new TextField(10);
        txtn2 = new TextField(10);
        txtR = new TextField(10);
        lbln1=new Label("Digite N1");
        lbln2=new Label("Digite N2");
        B = new Button("=");
    }
    public static void main(String z[])
    {
        APP form = new APP();
        form.setLayout(new FlowLayout());
        form.resize(400,400);
        form.setTitle("Minha Aplicacao");
        form.show();
    }
}
```

Passo 3: Adicionamos os objectos na janela (Frame)

Até agora os objectos apenas foram adicionados e inicializados na classe APP, contudo ainda não foram adicionados na janela (Frame) e ao fazermos a execução deste programa ele continuará mostrando uma janela simples como a anterior. Sendo assim, usa-se o método **add()** no construtor para adicionar cada um dos objectos na classe. Ora vejamos,

```
import java.awt.*;
public class ABC extends Frame
{
    TextField txtnome;
    Label lblnome,l2;
    Button ok;
    Choice ch;
    CheckboxGroup genero;
    Checkbox foot,bask,masc,fem;
    List L;
    public ABC()
    {
        txtnome=new TextField(10);
        genero=new CheckboxGroup();
        foot=new Checkbox("Futebol",true);
        bask=new Checkbox("Basketbol");
        masc=new Checkbox("Masculino",genero,false);
        fem=new Checkbox("Feminino",genero,true);
        ch=new Choice();
        L=new List();
        L.add("Maputo");
        L.add("Gaza");
        l2=new Label("Lista de cidades");
        lblnome=new Label("Digite teu nome");
        ok=new Button("OK");
        ch.add("Maputo");
        ch.add("Sofala");
        add(lblnome);
        add(txtnome);
        add(ok);
        add(ch);
        add(foot);
        add(bask);
        add(masc);
        add(fem);
        add(l2);
        add(L);
        setLayout(new FlowLayout());
    }
    public static void main(String j[])
    {
        ABC obj=new ABC();
        obj.setTitle("Meu programa");
        obj.resize(600,400);
        obj.show();
    }
}
```

}

1.2 Melhorando o visual com o pacote swing

Java possui uma extensão de classes que permitem desenvolver aplicações com o visual melhorado. Estas classes pertencem ao pacote `javax.swing` que contém novas classes como o:

- `JTextField` (substitui `TextField`)
- `JButton` (substitui o `Button`)
- `JComboBox` (substitui o `Choice`)
- `JCheckBox` (substitui o `Checkbox`)
- `JList` (substitui o `List`)
- `JFrame` (substitui o `Frame`)
- etc

Portanto, a única coisa a fazer é converter a aplicação `awt` para `swing` acrescentando o `J` a cada um dos objectos (para alguns objectos este padrão não é aplicável)

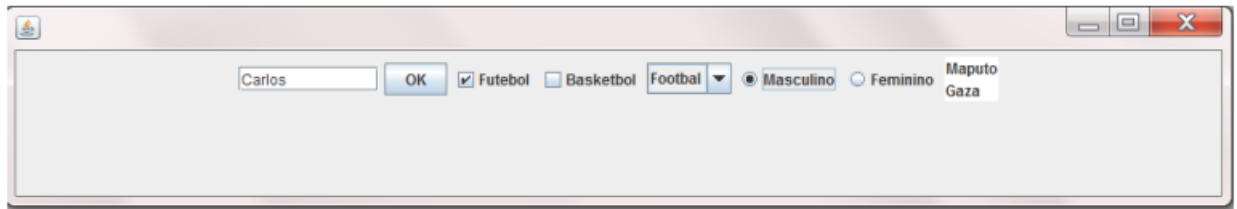
Vejamos o mesmo programa usando agora o `swing`:

```
import java.awt.*;
import javax.swing.*;
class APP extends JFrame{
    JTextField txt1;
    JCheckBox c,c2;
    JRadioButton m,f;
    JList l;
    JComboBox ch;
    JButton B;
    public APP()
    {
        m=new JRadioButton("Masculino");
        f=new JRadioButton("Masculino");
        txt1=new JTextField(10);
        c=new JCheckBox("Futebol");
        c2=new JCheckBox("Basketbol");
        B=new JButton("OK");
        ch=new JComboBox();
        ch.addItem("Footbal");
        ch.addItem("Basket")
        String[] lista={"Maputo","Gaza"};
        l=new JList(lista);
        add(txt1);
        add(B);
        add(c);
        add(c2);
        add(ch);
        add(m);
        add(f);
        add(l);
    }
}
```

Introdução à *Programação Orientada a Objectos*

```
public static void main(String args[])
{
    APP obj = new APP();
    obj.setLayout(new FlowLayout());
    obj.resize(300,300);
    obj.show();
}
}
```

Como resultado teremos o seguinte formulário:



The image shows a Java Swing window with a title bar containing a small icon, a maximize button, and a close button. The window's content area contains a form with the following elements: a text input field with the text 'Carlos'; an 'OK' button; a checked checkbox labeled 'Futebol'; an unchecked checkbox labeled 'Basketbol'; a dropdown menu currently showing 'Football'; a selected radio button labeled 'Masculino'; an unselected radio button labeled 'Feminino'; and a label 'Maputo Gaza' on the right side.

Conforme pudemos vêr, o pacote swing oferece uma melhoria significativa no visual e possui muitos outros recursos como:

- Adição de imagens aos objectos
- Diversos tipos de themes (Layouts)
- Muitos novos objectos.

LIÇÃO 2: ADIÇÃO DOS EVENTOS AOS OBJECTOS

2.1. Introdução

Em aplicações GUI os movimentos do mouse são controlados pelo sistema operativo, pois é ele quem decide que programa está ligado ao tal evento (click, duplo click, etc). Sendo assim, quando fazemos o click do mouse o sistema operativo armazena as coordenadas do cursor e estabelece a comunicação com o tal programa. O sinal que o programa recebe chamamos de evento. Em java os eventos são gerados através das classe que estão no pacote **java.awt.event**. Cada uma das classes está ligada a outras classes (interfaces) chamadas Listeners. Os Listeners conectam o objecto em causa (botão por exemplo) com o tal evento.

Objectivo geral da aula/Meta



Assim, no fim desta aula o aluno deve ser capaz de:

- Compreender os principais eventos usados em POO
- Adicionar um evento a uma aplicação usando um dos eventos discutidos

2.2 As principais classes (eventos) disponíveis em Java

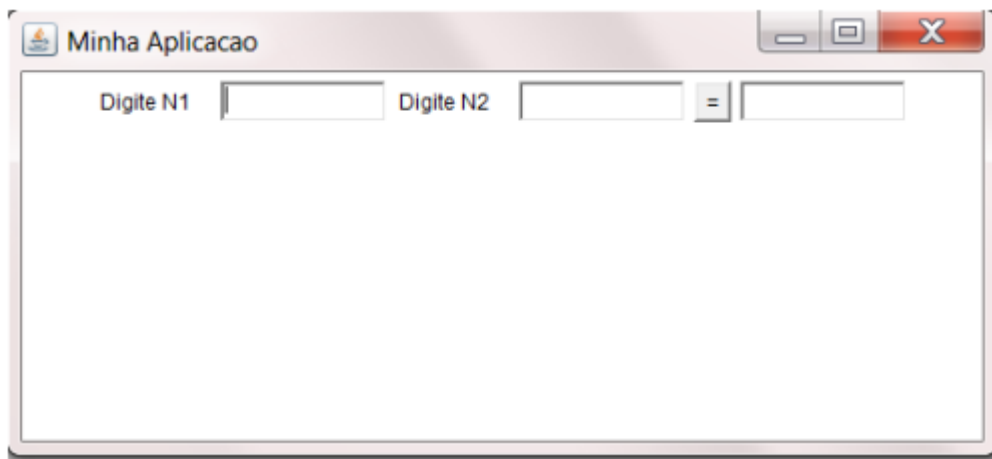
A seguit iremos apresentar uma tabela contendo as classes, a sua descrição e as interfaces a elas relacionadas na adição de eventos:

Evento	Descrição	Listener	Métodos obrigatórios
ActionEvent	Clique sobre um objecto	ActionListener	void actionPerformed(ActionEvent x){}
TextEvent	Ao escrever na caixa de texto	TextListener	void textValueChanged(TextEvent e){}
KeyEvent	Ao accionar qualquer tecla no teclado	KeyListener	void keyPressed(KeyEvent x){} void keyReleased(KeyEvent x){} void keyTyped(KeyEvent x){}
MouseEvent	Ao accionar o mouse sobre o objecto	MouseListener	void mousePressed(MouseEvent e){} void mouseReleased(MouseEvent e){} void mouseEntered(MouseEvent e){} void mouseExited(MouseEvent e){} void mouseClicked(MouseEvent e){}
MouseEvent	Ao mover o mouse	MouseEvent Listener	void mouseMoved(MouseEvent x) void mouseDragged(MouseEvent x)
WindowEvent	Qualquer acção sobre a janela do formulário.		void windowOpened(WindowEvent x); void windowClosing(WindowEvent x); void windowClosed(WindowEvent x); void windowIconified(WindowEvent x); void windowDeiconified(WindowEvent x); void windowActivated(WindowEvent x); void windowDeactivated(WindowEvent x);

Existem outras tantas classes usadas para outros tipos de eventos, contudo, o mais importante é escolher o evento certo para cada objecto. Por exemplo, usa-se o ActionEvent (e o seu ActionListener) para os botões.

3.1 Como adicionar eventos aos objectos

Na calculadora desenhada anteriormente, o botão “=” não realizava qualquer acção. Sendo assim, pretende-se que ao acciona-lo o sistema deve somar o conteúdo das caixas de texto **txtN1** e **txtN2** e armazenar o resultado em **txtR**.



O primeiro passo consiste em escolher o evento para o botão. Como o botão deve responder ao click do mouse, temos duas opções:

- **MouseListener** : possui o método `void mouseClicked(MouseEvent e){}` que é accionado quando se efectua o click sobre o mouse.
- **ActionListener**: possui o método `void actionPerformed(ActionEvent x){}` que também é accionado quando o click do mouse é efectuado.



Ao escolher o listener é obrigatório chamar todos os métodos obrigatórios (override), por mais que não façam nada.

Vamos ao programa:

Passo 1: Adicionar o evento na classe

Note que é obrigatório chamar o método `actionPerformed`, pois é neste método onde será realizada a soma.

```
import java.awt.*;
import java.awt.event.*;
class APP extends Frame implements ActionListener
{ TextField txtn1,txtn2,txtR;
  Label lbln1,lbln2;
  Button B;
  public APP()
  {
    txtn1 = new TextField(10);
    txtn2 = new TextField(10);
    txtR = new TextField(10);
    lbln1=new Label("Digite N1");
    lbln2=new Label("Digite N2");
    B = new Button("=");
    add(lbln1);
    add(txtn1);
    add(lbln2);
    add(txtn2);
    add(B);
    add(txtR);
  }
  public void actionPerformed(ActionEvent e){
  }
  public static void main(String z[])
  { APP form = new APP();
    form.setLayout(new FlowLayout());
```

```
form.resize(400,400);
form.setTitle("Minha Aplicacao");
form.show();
}
}
```

Passo 2: Adicionar o Listener no botão

Usamos o método `addActionListener()` que o botão possui para adicionar a acção click no botão:

```
import java.awt.*;
import java.awt.event.*;
class APP extends Frame implements ActionListener
{ TextField txtn1,txtn2,txtR;
  Label lbln1,lbln2;
  Button B;
  public APP()
  {
    txtn1 = new TextField(10);
    txtn2 = new TextField(10);
    txtR = new TextField(10);
    lbln1=new Label("Digite N1");
    lbln2=new Label("Digite N2");
    B = new Button("=");
    add(lbln1);
    add(txtn1);
    add(lbln2);
    add(txtn2);
    add(B);
    add(txtR);
    B.addActionListener(this);
  }
  public void actionPerformed(ActionEvent e){
  }
  public static void main(String z[])
  { APP form = new APP();
    form.setLayout(new FlowLayout());
    form.resize(400,400);
    form.setTitle("Minha Aplicacao");
    form.show();
  }
}
```

Passo 3: Programamos a soma no método `actionPerformed`

Agora que o botão B possui o listener quando accionarmos o clique no botão tudo que estiver dentro do método **actionPerformed** será executado. Daí, o próximo passo consiste em programar a calculadora neste método:

```
import java.awt.*;
import java.awt.event.*;
class APP extends Frame implements ActionListener
{ TextField txtn1,txtn2,txtR;
  Label lbln1,lbln2;
  Button B;
  public APP()
  {
    txtn1 = new TextField(10);
    txtn2 = new TextField(10);
    txtR = new TextField(10);
    lbln1=new Label("Digite N1");
    lbln2=new Label("Digite N2");
    B = new Button("=");
    add(lbln1);
    add(txtn1);
    add(lbln2);
    add(txtn2);
    add(B);
    add(txtR);
    B.addActionListener(this);
  }
  public void actionPerformed(ActionEvent e){
    int v1 = Integer.parseInt(txtn1.getText());
    int v2 = Integer.parseInt(txtn2.getText());
    int R=v1+v2;
    txtR.setText(Integer.toString(R));
  }
  public static void main(String z[])
  { APP form = new APP();
    form.setLayout(new FlowLayout());
    form.resize(400,400);
    form.setTitle("Minha Aplicacao");
    form.show();
  }
}
```

Nota-se o uso dos métodos de conversão de inteiro para String e vice-versa.

- Integer.toString() Converte um dado qualquer para String (pois as caixas de texto só recebem textos)
- Integer.parseInt() Converte um dado qualquer para int

Nota-se também o uso dos métodos das caixas de textos

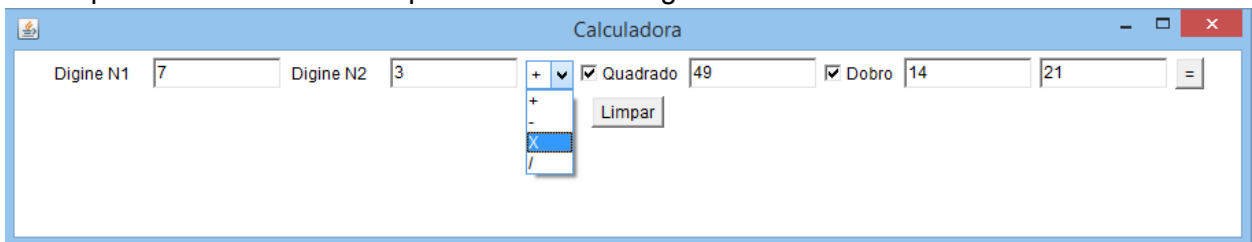
- `getText()` Busca o conteúdo existente na caixa de texto
- `setText()` Armazena um conteúdo qualquer na caixa de texto

Sendo assim o programa já está pronto:



Actividade de auto aprendizagem

Use o pacote AWT ou SWING para desenhar o seguinte formulário:



- Ao accionar no botão "=", o sistema calcula a operação entre N1 e N2 de acordo com a opção escolhida nas listas (+, -, X ou /).
- Ao accionar a check box Quadrado o sistema calcula o quadrado de N1
- Ao accionar a checkbox Dobro o sistema calcula o dobro de N1
- Ao accionar o botão "limpar", o sistema esvazia as caixas de texto N1 e N2.

Resultado

<pre> import java.awt.*; import java.awt.event.*; public class Prog1 extends Frame implements ActionListener,ItemListener{ TextField txtn1,txtn2,txtquad,txtdobro,txtr; Checkbox chkq,chkd; Label l1,l2; Choice ch; Button b,clr; public Prog1(){ l1=new Label("Digue N1"); l2=new Label("Digue N2"); txtn1 = new TextField(10); txtn2 = new TextField(10); txtquad = new TextField(10); txtdobro = new TextField(10); txtr = new TextField(10); chkq = new Checkbox("Quadrado"); chkq.addItemListener(this); chkd = new Checkbox("Dobro"); chkd.addItemListener(this); b = new Button("="); clr = new Button("Limpar"); b.addActionListener(this); clr.addActionListener(this); ch=new Choice();ch.add("+");ch.add("-");ch.add("X");ch.add("/"); add(l1);add(txtn1);add(l2);add(txtn2);add (ch);add(chkq);add(txtquad);add(chkd);add(txtdo bro);add(txtr); add(b);add(clr); </pre>	<pre> public void actionPerformed(ActionEvent g){ if(g.getSource()==b) { int n1 = Integer.parseInt(txtn1.getText()); int n2 = Integer.parseInt(txtn2.getText()); String sel = ch.getSelectedItem(); int R=0; if(sel.equals("+")){ R=n1+n2; } else if(sel.equals("-")){ R=n1-n2; } else if(sel.equals("X")){ R=n1*n2; } else if(sel.equals("/")){ R=n1/n2; } txtr.setText(Integer.toString(R)); } else if(g.getSource()==clr){ txtn1.setText(""); txtn2.setText(""); } } public void itemStateChanged(ItemEvent e){ </pre>
--	---

Introdução à Programação Orientada a Objectos

<pre>} public static void main(String g[]) { Prog1 form=new Prog1(); form.setLayout(new FlowLayout()); form.resize(400,400); form.setTitle("Calculadora"); form.setVisible(true); } //Continua na outra coluna --7</pre>	<pre> if(e.getSource()==chkq) { int n1 = Integer.parseInt(txtn1.getText()); int R=n1*n1; txtquad.setText(Integer.toString(R)); } else if(e.getSource()==chkd){ int n1 = Integer.parseInt(txtn1.getText()); int R=n1*2; txtdobro.setText(Integer.toString(R)); } } }</pre>
---	--

[Fim da Unidade VI]

Bibliografia utilizada

- Oracle Java Documentation The Java TM Tutorials

<http://docs.oracle.com/javase/tutorial/java>

Site visitado aos 20 de Janeiro de 2018 pelas 23:40

- Balagurusamy, E. Programming with Java a primer. 4th Edition. Tata McGraw-Hill. 2010
- Buyya, Rajkumar. Object-oriented Programming with Java: Essentials and Applications. 2009
- Harvey M. Deitel. Java: Como Programar. 6 ed. São Paulo: Pearson education do Brasil, 2005
- Naughton, Patrick, Dominando o Java, Guia Autorizado da Sun Microsystems, Editora Makron Books, 1997.